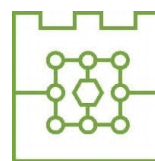




**Politechnika Krakowska
im. Tadeusza Kościuszki**
Wydział Informatyki i Telekomunikacji



Michał Bąk

Numer albumu: 144753

Porównanie metod uczenia przez wzmacnianie w Ultimate Texas Hold'em z niepełną informacją: wpływ reprezentacji stanu i funkcji nagrody na jakość strategii

Comparison of reinforcement learning methods in Ultimate Texas Hold'em under imperfect information: the impact of state representation and reward function on strategy quality

**Praca dyplomowa magisterska
na kierunku Informatyka**

Praca wykonana pod kierunkiem:
mgr inż. Piotr Biskup

Kraków, 2026

SPIS TREŚCI

1	Wstęp	7
1.1	Cel pracy	7
1.2	Zakres pracy	7
1.3	Problem badawczy	8
1.4	Struktura pracy	8
2	Podstawy teoretyczne	11
2.1	Poker jako gra z niepełną informacją	11
2.2	Texas Hold'em	11
2.3	Ranking układów pokerowych	11
2.4	Ultimate Texas Hold'em	12
2.4.1	Charakterystyka gry	12
2.4.2	Różnice względem klasycznego Texas Hold'em	12
2.4.3	Przebieg rozdania	12
2.4.4	Zakłady i decyzje gracza	13
2.4.5	Rozstrzyganie rozdania	13
2.4.6	Zakłady poboczne	14
2.4.7	Tabele wypłat	14
2.5	Wartość oczekiwana i przewaga kasyna	15
2.6	Strategie bazowe i ocena jakości strategii	16
2.7	Uczenie przez wzmacnianie	16
2.7.1	Podstawowe pojęcia	16
2.8	Modelowanie UTH jako procesu decyzyjnego z częściową obserwowalnością	17
3	Projekt i implementacja środowiska Ultimate Texas Hold'em	19
3.1	Założenia projektowe środowiska	19
3.2	Architektura rozwiązania	20
3.2.1	Przepływ danych podczas wykonania akcji	21
3.3	Model kart i sterowanie talią	22
3.3.1	Reprezentacja karty	22
3.3.2	Standardowa talia	23
3.3.3	Abstrakcja źródła kart	23

3.3.4	Deterministyczna talia testowa	23
3.3.5	Powtarzalność losowych rozdań	24
3.4	Ocena układów pokerowych	24
3.4.1	Reprezentacja wartości ręki	24
3.4.2	Ocena układu pięciokartowego	25
3.4.3	Wybór najlepszej ręki	25
3.5	Stan wewnętrzny i obserwacja agenta	26
3.5.1	Pełny stan rozdania	27
3.5.2	Obserwacja agenta	28
3.5.3	Projekcja stanu na obserwację	29
3.5.4	Ochrona przed wyciekiem informacji	30
3.6	Maszyna stanów i przestrzeń akcji	30
3.6.1	Pojedynczy zakład Play i walidacja decyzji	31
3.7	Showdown i rozliczanie zakładów	32
3.7.1	Przebieg showdownu	32
3.7.2	Kwalifikacja krupiera	33
3.7.3	Model rozliczenia zakładów	34
3.7.4	Rozliczenie zakładu Blind	34
3.7.5	Rozliczenie Ante i Play	34
3.7.6	Rozliczenie spasowania	35
3.7.7	Przykładowe rozliczenie	35
3.8	Funkcja nagrody i wynik epizodu	36
3.9	Interfejs silnika	36
3.9.1	Tworzenie silnika	36
3.9.2	Rozpoczęcie epizodu	37
3.9.3	Wykonanie decyzji	37
3.9.4	Właściwości publiczne	37
3.10	Rejestrowanie przebiegu i walidacja środowiska	38
3.10.1	Opcjonalne rejestrowanie historii	38
3.10.2	Rekordy historii i ich transakcyjny zapis	38
3.10.3	Poziomy testowania	39
3.10.4	Pokrycie kodu	39
3.11	Ograniczenia środowiska	40
3.12	Podsumowanie	40
4	Agenci bazowi i infrastruktura eksperymentalna	43
4.1	Wspólny interfejs agenta	43
4.2	Prowadzenie pojedynczego epizodu	43
4.3	Agent losowy	44

4.3.1	Rozdzielenie źródeł losowości	44
4.3.2	Walidacja implementacji	44
4.4	Agent regułowy	44
4.4.1	Decyzja przed flopem	44
4.4.2	Decyzja po flopie	45
4.4.3	Decyzja po odkryciu wszystkich kart wspólnych	45
4.4.4	Implementacja strategii regułowej	45
4.5	Symulacja wielu epizodów	46
4.6	Powtarzalność eksperymentów	46
4.7	Metryki oceny	47
4.8	Porównanie agentów bazowych	47
4.9	Zapis wyników eksperymentu	48
4.10	Walidacja infrastruktury eksperymentalnej	48
4.11	Podsumowanie	49
5	Metody uczenia przez wzmacnianie	51
5.1	Reprezentacja stanu	51
5.1.1	Wspólny interfejs kodowania	51
5.2	Surowa reprezentacja obserwacji	52
5.2.1	Kodowanie kart	52
5.2.2	Kodowanie fazy rozdania	52
5.2.3	Maska legalnych akcji	53
5.2.4	Wymiar reprezentacji	53
5.2.5	Charakter reprezentacji surowej	53
5.3	Reprezentacja wzbogacona o cechy domenowe	54
5.3.1	Cechy kart prywatnych	54
5.3.2	Kategoria układu pokerowego	54
5.3.3	Cechy strukturalne	55
5.3.4	Wymiar reprezentacji wzbogaconej	56
5.4	Porównanie wariantów reprezentacji	56
5.5	Konfiguracja reprezentacji	56
5.6	Walidacja reprezentacji	57
5.7	Hipoteza dotycząca reprezentacji stanu	57
5.8	Funkcja nagrody	57
5.8.1	Terminalny charakter nagrody	58
5.9	Nagroda oparta na zysku netto	58
5.9.1	Dokładność obliczeń finansowych	58
5.10	Nagroda skalowana względem maksymalnej stawki	59
5.10.1	Znaczenie skalowania	59

5.11	Porównanie wariantów funkcji nagrody	60
5.12	Konfiguracja funkcji nagrody	60
5.13	Walidacja funkcji nagrody	61
5.14	Hipoteza dotycząca funkcji nagrody	61
5.15	Deep Q-Network	61
5.15.1	Architektura sieci Q	62
5.15.2	Stałe odwzorowanie akcji	62
5.15.3	Maskowanie nielegalnych akcji	62
5.15.4	Strategia epsilon-greedy	63
5.15.5	Bufor doświadczeń	63
5.15.6	Sieć docelowa i cel Bellmana	64
5.15.7	Funkcja straty i optymalizacja	64
5.15.8	Przebieg treningu	65
5.15.9	Konfiguracja treningu	65
5.15.10	Reprodukowalność treningu	66
5.15.11	Zapisywanie modeli	66
5.15.12	Macierz wariantów DQN	66

1. Wstęp

Od zawsze interesowały mnie gry karciane jako obszar zastosowania metod sztucznej inteligencji, ponieważ łączą w sobie elementy losowości oraz podejmowania decyzji przy niepełnej informacji. W klasycznych problemach uczenia maszynowego model uczy się na podstawie gotowego zbioru danych, a w grach agent musi samodzielnie podejmować decyzje, obserwować ich konsekwencje i poprawiać swoją strategię.

Szczególnie interesującym mnie przypadkiem są gry pokerowe, w których wynik pojedynczego rozdania nie zależy jedynie od posiadanych kart, ale również od decyzji podjętych w ramach kolejnych etapów gry. Chciałbym zwrócić uwagę na Ultimate Texas Hold'em — kasynową odmianę pokera bazującą na zasadach Texas Hold'em, w której gracz rywalizuje nie z innymi graczami, a z krupierem. Odmiana ta ma ograniczoną liczbę możliwych akcji, jasno określone reguły wypłat oraz sekwencyjny przebieg rozdania. Te cechy, w moim mniemaniu, czynią ją odpowiednim problemem do rozważenia w ramach uczenia przez wzmacnianie.

Uczenie przez wzmacnianie (ang. *reinforcement learning*, RL) pozwala analizować sytuacje, w których agent uczy się strategii poprzez interakcję ze środowiskiem. W przypadku Ultimate Texas Hold'em środowiskiem może być symulator gry, obserwacją informacje dostępne graczowi w bieżącym etapie rozdania, akcją decyzja gracza, natomiast nagrodą końcowy wynik finansowy rozdania. Takie podejście umożliwia badanie, w jaki sposób różne reprezentacje stanu oraz różne definicje funkcji nagrody wpływają na jakość wyuczonej strategii.

1.1. Cel pracy

Głównym celem tej pracy jest ocena przydatności wybranych metod uczenia przez wzmacnianie do podejmowania decyzji w Ultimate Texas Hold'em. Analizie podlega również wpływ sposobu reprezentacji obserwacji oraz konstrukcji funkcji nagrody na przebieg uczenia i jakość otrzymywanej strategii.

Za cel praktyczny obrałem zaprojektowanie i implementację kompletnego procesu badawczego obejmującego środowisko gry, agentów bazowych i uczących się, mechanizm prowadzenia treningu oraz powtarzalną ocenę uzyskanych strategii. Porównanie obejmuje zarówno różnice pomiędzy metodami uczenia, jak i ich odniesienie do strategii bazowych, w tym agenta losowego i agenta regułowego.

1.2. Zakres pracy

Zakres pracy obejmuje zaprojektowanie i implementację środowiska odwzorowującego podstawowy wariant Ultimate Texas Hold'em, w którym uwzględniłem podstawowe elementy rozgrywki: karty gracza i krupiera, karty wspólne, obowiązkowe zakłady Ante

i Blind oraz decyzyjny zakład Play. W mojej wersji środowiska pominąłem zakłady poboczne, takie jak Trips oraz Progressive, ponieważ nie są one bezpośrednio związane z głównym procesem decyzyjnym agenta.

W ramach pracy przygotowałem różne warianty numerycznej reprezentacji obserwacji agenta, utworzonej na podstawie dostępnej części stanu gry. Reprezentacja obserwacji obejmuje między innymi informacje o kartach gracza, kartach wspólnych, aktualnym etapie rozdania oraz dostępnych akcjach. Podałem analizie również wpływ sposobu przekształcania finansowego wyniku rozdania w sygnał nagrody.

Założenia niniejszej pracy są następujące:

1. Implementacja środowiska odwzorowującego podstawowy wariant UTH.
2. Implementacja mechanizmu oceny układów, przebiegu rozdania i rozliczania zakładów.
3. Przygotowanie agentów bazowych (ang. *baseline agents*) stanowiących punkty odniesienia dla metod uczących się.
4. Implementacja wybranych metod uczenia przez wzmacnianie.
5. Przygotowanie i porównanie różnych reprezentacji obserwacji.
6. Przygotowanie i porównanie wybranych funkcji nagrody.
7. Opracowanie powtarzalnej procedury treningu i oceny agentów.
8. Analiza jakości strategii, przebiegu uczenia oraz wyników uzyskanych względem pozostałych metod i strategii bazowych.

1.3. Problem badawczy

Problem badawczy, który postanowiłem rozważyć, dotyczy wpływu wyboru metody uczenia przez wzmacnianie, reprezentacji obserwacji oraz funkcji nagrody na przebieg uczenia oraz jakość strategii uzyskiwanej w Ultimate Texas Hold'em.

Analizie poddałem także to, które konfiguracje pozwalają osiągnąć najwyższy średni wynik netto i najmniejszą oczekiwaną stratę, jak stabilny jest proces uczenia oraz czy uzyskane strategie przewyższają przyjęte strategie bazowe. Agenci bazowi służą jako punkty odniesienia do oceny metod uczenia przez wzmacnianie.

1.4. Struktura pracy

Struktura pracy składa się z kilku rozdziałów. W rozdziale pierwszym przedstawiam cel, zakres oraz problem badawczy pracy. W rozdziale drugim zawieram podstawy teoretyczne dotyczące pokera, Texas Hold'em, Ultimate Texas Hold'em, wartości oczekiwanej, przewagi kasyna oraz uczenia przez wzmacnianie.

W kolejnych rozdziałach opisuję projekt środowiska symulacyjnego, sposób reprezentacji stanu gry, funkcję nagrody oraz zastosowane metody uczenia przez wzmacnianie. Następnie przedstawiam eksperymenty, uzyskane wyniki oraz analizę porównawczą agentów. Ostatni rozdział zawiera podsumowanie pracy, wnioski oraz możliwe kierunki dalszego rozwoju.

2. Podstawy teoretyczne

2.1. Poker jako gra z niepełną informacją

Poker jest rodziną gier karcianych, w których wynik zależy zarówno od losowego rozdania kart, jak i od decyzji podejmowanych przez graczy. W przeciwieństwie do gier z pełną informacją, takich jak szachy, uczestnicy rozgrywki nie znają wszystkich elementów stanu gry. Ukryte pozostają przede wszystkim karty przeciwników, co sprawia, że decyzje podejmowane są w warunkach niepewności.

Z punktu widzenia sztucznej inteligencji poker jest interesującym problemem badawczym, ponieważ łączy losowość, niepełną informację, sekwencyjne podejmowanie decyzji oraz konieczność maksymalizacji wyniku w długim horyzoncie czasowym. W takim środowisku pojedyncza decyzja nie powinna być oceniana wyłącznie przez pryzmat wyniku jednego rozdania, lecz przez jej wpływ na oczekiwaną wartość strategii w wielu powtórzeniach gry.

2.2. Texas Hold'em

Texas Hold'em jest jedną z najpopularniejszych odmian pokera. Każdy gracz otrzymuje dwie zakryte karty własne (ang. *hole cards*), natomiast na stole stopniowo odkrywanych jest pięć kart wspólnych (ang. *community cards*). Gracz tworzy najlepszy możliwy układ pięciokartowy, wykorzystując dowolną kombinację swoich dwóch kart oraz pięciu kart wspólnych.

Rozgrywka w klasycznym Texas Hold'em składa się z kilku etapów: etapu przed flopem (ang. *preflop*), flopa (ang. *flop*), czyli odkrycia trzech pierwszych kart wspólnych, turna (ang. *turn*), czyli czwartej karty wspólnej, rivera (ang. *river*), czyli piątej i ostatniej karty wspólnej, oraz showdownu (ang. *showdown*), czyli końcowego porównania układów. W trakcie kolejnych rund licytacji gracze mogą podejmować decyzje takie jak czekanie (ang. *check*), postawienie zakładu (ang. *bet*), sprawdzenie zakładu przeciwnika (ang. *call*), podbicie (ang. *raise*) lub spasowanie (ang. *fold*). W odmianach no-limit możliwe jest również stosowanie różnych rozmiarów zakładów, co istotnie zwiększa przestrzeń możliwych strategii.

2.3. Ranking układów pokerowych

W Texas Hold'em oraz Ultimate Texas Hold'em wynik rozdania zależy od siły najlepszego układu pięciokartowego utworzonego z dostępnych kart. Układy pokerowe mają ustaloną hierarchię, która pozwala jednoznacznie porównać rękę gracza i krupiera podczas showdownu. Przykładowy ranking układów od najsilniejszego do naj słabszego przedstawiono w tabeli 2.1 [1].

Najsilniejszym układem jest poker królewski, natomiast naj słabszym układem jest

wysoka karta. W przypadku gdy dwóch uczestników rozdania posiada układ tej samej kategorii, o zwycięstwie decydują wartości kart wchodzących w skład układu oraz karty dodatkowe, określane jako kickery.

Tabela 2.1. Ranking układów pokerowych od najsilniejszego do najslabszego

Układ	Opis
Poker królewski	A, K, Q, J, 10 w tym samym kolorze
Poker	Pięć kolejnych kart w tym samym kolorze
Kareta	Cztery karty tej samej wartości
Full	Trójka oraz para
Kolor	Pięć kart w tym samym kolorze
Strit	Pięć kolejnych kart
Trójka	Trzy karty tej samej wartości
Dwie pary	Dwa różne układy par
Para	Dwie karty tej samej wartości
Wysoka karta	Brak silniejszego układu

2.4. Ultimate Texas Hold'em

2.4.1. Charakterystyka gry

Ultimate Texas Hold'em jest kasynową odmianą pokera opartą na zasadach Texas Hold'em. W przeciwieństwie do klasycznej odmiany wieloosobowej gracz nie rywalizuje z innymi graczami, lecz z krupierem reprezentującym kasyno. Zarówno gracz, jak i krupier otrzymują po dwie karty własne, a następnie korzystają z pięciu kart wspólnych w celu utworzenia najlepszego możliwego układu pięciokartowego [1], [2].

2.4.2. Różnice względem klasycznego Texas Hold'em

Najważniejsza różnica względem klasycznego Texas Hold'em polega na tym, że w Ultimate Texas Hold'em gracz podejmuje decyzje przeciwko kasynu, a nie przeciwko innym graczom. Nie występuje więc klasyczna rywalizacja między wieloma uczestnikami stołu, nie ma konieczności blefowania przeciwników ani reagowania na ich indywidualne style gry.

Istotną cechą Ultimate Texas Hold'em jest również ograniczona liczba momentów decyzyjnych. Gracz może wykonać zakład Play (ang. *Play bet*) tylko raz w trakcie rozdania: przed flopem, po flopie albo po odkryciu wszystkich kart wspólnych. Im wcześniej gracz zdecyduje się na zagranie agresywne, tym większy zakład może postawić, ale decyzja jest wtedy podejmowana przy mniejszej liczbie informacji [1], [3].

2.4.3. Przebieg rozdania

Rozdanie rozpoczyna się od postawienia przez gracza obowiązkowych zakładów Ante (ang. *Ante bet*) oraz Blind (ang. *Blind bet*). Zakłady te mają zwykle taką samą wartość. Opcjonalnie gracz może również postawić zakład poboczny (ang. *side bet*), na przykład Trips, który jest rozliczany niezależnie od wyniku pojedynku z krupierem [1].

Tabela 2.2. Porównanie Texas Hold'em i Ultimate Texas Hold'em

Cecha	Texas Hold'em	Ultimate Texas Hold'em
Przeciwnik	inni gracze	krupier / kasyno
Liczba graczy	wielu	jeden gracz przeciwko krupierowi
Zakłady	zmienne	ustalone mnożniki Ante
Blefowanie	istotne	brak klasycznego blefowania
Liczba decyzji	duża	ograniczona

Po wniesieniu zakładów gracz oraz krupier otrzymują po dwie zakryte karty własne. Następnie gracz podejmuje pierwszą decyzję. Może zaczekać albo wykonać zakład Play w wysokości trzykrotności lub czterokrotności zakładu Ante.

Jeżeli gracz nie wykona zakładu Play przed flopem, odkrywane są trzy pierwsze karty wspólne, czyli flop. Na tym etapie gracz może ponownie zaczekać albo wykonać zakład Play w wysokości dwukrotności Ante. Jeżeli również po flopie gracz zdecyduje się zaczekać, odkrywane są dwie ostatnie karty wspólne, czyli turn oraz river. Wtedy gracz musi wybrać pomiędzy spasowaniem a wykonaniem zakładu Play w wysokości jednokrotności Ante [1], [3].

2.4.4. Zakłady i decyzje gracza

W podstawowym modelu gry występują trzy główne rodzaje zakładów: Ante, Blind oraz Play. Zakłady Ante i Blind są obowiązkowe i stawiane przed rozpoczęciem rozdania. Zakład Play jest natomiast bezpośrednio związany z decyzją gracza i może zostać wykonany tylko raz w trakcie rozdania.

Z punktu widzenia modelowania problemu jako środowiska uczenia przez wzmocnianie najważniejszy jest zakład Play, ponieważ jego wysokość i moment wykonania wynikają z decyzji agenta. Ante i Blind można traktować jako początkowy koszt udziału w rozdaniu, natomiast Play odzwierciedla wybór strategii w aktualnym stanie gry.

Dostępne decyzje gracza w poszczególnych etapach rozdania przedstawiono w tabeli 2.3.

Tabela 2.3. Dostępne decyzje gracza w Ultimate Texas Hold'em

Etap	Dostępne decyzje	Wysokość zakładu Play
Preflop	czekanie lub zakład Play	3x albo 4x Ante
Flop	czekanie lub zakład Play	2x Ante
Po odkryciu całego stołu	spasowanie lub zakład Play	1x Ante

2.4.5. Rozstrzygnięcie rozdania

Po zakończeniu etapu decyzyjnego krupier odsłania swoje dwie karty własne. Zarówno gracz, jak i krupier tworzą najlepszy możliwy układ pięciokartowy z siedmiu dostępnych kart: dwóch kart własnych oraz pięciu kart wspólnych. Następnie układy są

porównywane zgodnie ze standardowym rankingiem układów pokerowych.

Krupier kwalifikuje się do gry, jeżeli posiada co najmniej parę. Jeżeli krupier się nie kwalifikuje, zakład Ante jest zwracany graczowi. Zakład Play jest rozliczany na podstawie bezpośredniego porównania układu gracza i krupiera. W przypadku zwycięstwa gracza Play wypłacany jest w stosunku 1:1, w przypadku przegranej gracz traci zakład Play, a w przypadku remisu zakład jest zwracany [1], [3].

Zakład Blind ma odrębną logikę rozliczania. Jeżeli gracz pokona krupiera układem o wartości co najmniej strita, Blind wypłacany jest zgodnie z tabelą wypłat. Jeżeli gracz wygra rozdanie układem słabszym niż strit, zakład Blind jest zwracany. W przypadku przegranej gracza zakład Blind jest tracony, a w przypadku remisu zwracany [1].

2.4.6. Zakłady poboczne

W wielu wariantach Ultimate Texas Hold'em dostępne są również zakłady poboczne, takie jak Trips lub Progressive. Zakład Trips jest opcjonalny i wypłacany na podstawie końcowego układu gracza, niezależnie od tego, czy gracz pokonał krupiera. Najczęściej wygrywa on wtedy, gdy końcowy układ gracza ma wartość co najmniej trójki [1].

Zakłady poboczne mogą różnić się w zależności od kasyna i stosowanej tabeli wypłat.

W zaimplementowanym środowisku nie uwzględniono zakładów pobocznych Trips ani Progressive. Wynika to z faktu, że głównym celem pracy jest modelowanie sekwencyjnego procesu decyzyjnego dotyczącego zakładu Play. Zakłady poboczne są opcjonalne, zależą od tabel wypłat konkretnego kasyna i zwiększają wariancję wyniku, nie zmieniając podstawowej struktury decyzji: czekanie, zakład Play albo spasowanie.

2.4.7. Tabele wypłat

Zakład Blind nie jest wypłacany dla każdego zwycięskiego układu gracza. Zgodnie z tabelą wypłat, bonus na zakładzie Blind wypłacany jest wtedy, gdy gracz pokona krupiera i posiada układ o wartości co najmniej strita. Jeżeli gracz pokona krupiera układem słabszym niż strit, zakład Blind jest zwracany. Tabelę wypłat zakładu Blind przedstawiono w tabeli 2.4 [1].

Tabela 2.4. Tabela wypłat zakładu Blind przyjęta w modelu środowiska

Układ gracza	Wypłata
Poker królewski	500:1
Poker	50:1
Kareta	10:1
Full	3:1
Kolor	3:2
Strit	1:1
Pozostałe układy	Zwrot

Zakład Trips oraz zakład Progressive są opcjonalnymi zakładami pobocznymi, rozliczanymi niezależnie od pojedynku z krupierem i zależnymi od tabel wypłat konkretnego kasyna. Ponieważ nie wpływają one na podstawową decyzję Play, w zaimplementowanym środowisku zakładów tych nie uwzględniono.

2.5. Wartość oczekiwana i przewaga kasyna

Wartość oczekiwana (ang. *expected value*, EV) określa średni wynik finansowy decyzji lub strategii przy założeniu dużej liczby powtórzeń gry. W kontekście Ultimate Texas Hold'em pojedyncze rozdanie może zakończyć się zarówno zyskiem, jak i stratą, jednak jakość strategii należy oceniać na podstawie średniego wyniku uzyskiwanego w wielu rozdaniach.

Przewaga kasyna (ang. *house edge*) jest matematyczną miarą określającą oczekiwany zysk kasyna względem zakładu gracza. Nie oznacza ona, że kasyno wygrywa każde pojedyncze rozdanie, lecz że przy dużej liczbie rozegranych rozdań średni wynik będzie przesunął się na korzyść kasyna. Gracz może więc osiągać krótkoterminowe zyski, jednak w długim okresie wynik gry powinien zbliżać się do wartości oczekiwanej wynikającej z zasad gry [4].

W Ultimate Texas Hold'em przewaga kasyna jest zwykle podawana względem początkowego zakładu Ante. Ponieważ gracz może dodatkowo zwiększyć swoje zaangażowanie poprzez zakład Play, obok klasycznej przewagi kasyna stosuje się również miarę określaną jako element ryzyka (ang. *element of risk*), która odnosi oczekiwaną stratę do średniej całkowitej kwoty zaangażowanej w rozdanie [3], [5]. Nawet niewielka procentowa przewaga prowadzi więc do istotnej oczekiwanej straty w długim horyzoncie czasowym.

Przykładowe wartości przewagi kasyna dla wybranych gier przedstawiono w tabeli 2.5 [5].

Tabela 2.5. Przykładowe wartości przewagi kasyna dla wybranych gier kasynowych

Gra	Przewaga kasyna
Blackjack	0,28%
Ultimate Texas Hold'em	2,19%
Ruletka europejska	2,70%
Ruletka amerykańska	5,26%
Automaty do gier	2–15%

Tabela 2.5 pokazuje, że przewaga kasyna jest cechą konstrukcyjną gier hazardowych, ale jej poziom istotnie różni się pomiędzy grami, a podane wartości zależą przy tym od wariantu zasad gry oraz przyjętego sposobu normalizacji. Celem niniejszej pracy nie jest wykazanie, że agent może trwale uzyskać dodatnią wartość oczekiwaną w grze zaprojektowanej na korzyść kasyna, lecz sprawdzenie, które metody uczenia przez wzmocnianie są w stanie nauczyć się strategii minimalizującej tę przewagę.

2.6. Strategie bazowe i ocena jakości strategii

Strategie bazowe stanowią punkty odniesienia służące do oceny agentów uczących się. Agent losowy wybiera jedną z legalnych akcji bez uwzględniania siły kart, stanowi prosty punkt odniesienia i umożliwia wstępną kontrolę działania infrastruktury eksperymentalnej. Agent regułowy wykorzystuje z góry określone heurystyki i reprezentuje bardziej wymagający punkt odniesienia.

Wyniki agentów uczących się powinny być porównywane zarówno pomiędzy sobą, jak i względem strategii bazowych. Samo przewyższenie agenta losowego nie jest wystarczającym dowodem uzyskania jakościowej strategii, natomiast porównanie z agentem regułowym pozwala ocenić, czy uczenie prowadzi do decyzji konkurencyjnych wobec jawnie zaprojektowanych zasad.

2.7. Uczenie przez wzmacnianie

Uczenie przez wzmacnianie (ang. *reinforcement learning*, RL) jest paradygmatem uczenia maszynowego, w którym agent uczy się podejmowania decyzji poprzez interakcję ze środowiskiem. W każdym kroku agent obserwuje aktualny stan (ang. *state*) środowiska, wybiera akcję (ang. *action*), a następnie otrzymuje nagrodę (ang. *reward*) oraz przechodzi do kolejnego stanu. Celem agenta jest wyuczenie strategii, określanej również jako polityka decyzyjna (ang. *policy*), która maksymalizuje sumę nagród w długim horyzoncie czasowym [6].

Z perspektywy uczenia przez wzmacnianie Ultimate Texas Hold'em jest interesującym środowiskiem, ponieważ decyzje gracza są sekwencyjne i podejmowane przy różnym poziomie dostępnej informacji. Przed flopem gracz zna jedynie swoje dwie karty, ale może wykonać najwyższy zakład Play. Po flopie posiada więcej informacji, lecz maksymalny zakład jest mniejszy. Po odkryciu wszystkich kart wspólnych zna już pełny board, ale może wykonać wyłącznie zakład 1x Ante albo spasować. Agent musi więc nauczyć się kompromisu pomiędzy ryzykiem, ilością informacji oraz potencjalną wypłatą.

2.7.1. Podstawowe pojęcia

Zachowanie agenta opisuje polityka π , która może być deterministyczna, przypisując stanowi jedną akcję, albo stochastyczna, określając rozkład prawdopodobieństwa wyboru akcji. Jakość decyzji ocenia się nie na podstawie pojedynczej nagrody, lecz zwrotu epizodycznego, czyli sumy nagród uzyskanych do końca epizodu:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}, \quad (2.1)$$

gdzie $\gamma \in [0, 1]$ jest współczynnikiem dyskontowania, a T oznacza koniec epizodu. Współczynnik γ reguluje wagę nagród odległych w czasie względem nagród natychmiastowych [6].

Oczekiwany zwrot z danego stanu opisuje funkcja wartości stanu $V^\pi(s)$, natomiast oczekiwany zwrot po wykonaniu akcji w danym stanie - funkcja wartości akcji $Q^\pi(s, a)$. Znajomość tych funkcji pozwala porównywać decyzje i stanowi podstawę wielu metod uczenia [6].

W trakcie uczenia agent musi równoważyć eksplorację, czyli wypróbowywanie nowych akcji w celu zdobycia informacji, oraz eksploatację, czyli wybór akcji uznanych dotąd za najlepsze. Niewłaściwy kompromis między nimi prowadzi do utknięcia w strategii suboptymalnej albo do niestabilnego przebiegu uczenia.

Funkcje wartości i polityki można reprezentować w postaci tablicowej, gdy przestrzeń stanów jest niewielka, albo za pomocą aproksymacji funkcji, na przykład sieci neuronowych, gdy przestrzeń stanów jest duża lub ciągła. Aproksymacja umożliwia generalizację pomiędzy podobnymi stanami, lecz może pogarszać stabilność i zbieżność uczenia [6].

W rozważanym problemie szczególne znaczenie mają dwa elementy konfiguracji procesu uczenia: sposób reprezentacji obserwacji, decydujący o tym, jakie informacje trafiają na wejście agenta, oraz konstrukcja funkcji nagrody, przekształcająca finansowy wynik rozdania w sygnał uczący. Oba te elementy są przedmiotem analizy w dalszej części pracy.

2.8. Modelowanie UTH jako procesu decyzyjnego z częściową obserwowalnością

W Ultimate Texas Hold'em agent nie obserwuje pełnego stanu środowiska, ponieważ nie zna kart krupiera, kart spalonych ani przyszłej kolejności talii. Podejmuje decyzję na podstawie obserwacji zawierającej własne karty, odkryte karty wspólne, bieżący etap rozdania oraz legalne akcje.

Problem można zatem traktować jako częściowo obserwowalny proces decyzyjny. Polityka agenta odwzorowuje dostępną obserwację na rozkład prawdopodobieństwa wyboru akcji, natomiast celem uczenia jest maksymalizacja oczekiwanego zwrotu uzyskiwanego w wielu epizodach. Rozróżnienie pełnego stanu środowiska od obserwacji agenta jest kluczowe dla zachowania niepełnej informacji charakterystycznej dla pokera i zostało bezpośrednio odwzorowane w projekcie środowiska opisanym w kolejnym rozdziale [6].

3. Projekt i implementacja środowiska Ultimate Texas Hold'em

W niniejszym rozdziale przedstawiono projekt oraz implementację środowiska Ultimate Texas Hold'em wykorzystywanego w dalszej części pracy do uczenia i oceny agentów. Opisane wcześniej zasady gry zostały odwzorowane w postaci powtarzalnego i testowalnego silnika, który zarządza przebiegiem rozdania, udostępnia agentowi legalne akcje oraz rozlicza wynik finansowy rozgrywki.

Szczególną uwagę poświęcono zachowaniu niepełnej informacji charakterystycznej dla pokera. Agent otrzymuje wyłącznie informacje, które byłyby dostępne rzeczywistemu graczowi, natomiast karty krupiera, karty spalone oraz kolejność pozostałych kart w talii pozostają elementami wewnętrznego stanu środowiska. Takie rozdzielanie pozwala ograniczyć ryzyko niezamierzonego ujawnienia informacji podczas uczenia modeli.

Implementację podzielono na niezależne moduły odpowiedzialne między innymi za reprezentację kart, ocenę układów, sterowanie przebiegiem rozdania, walidację akcji oraz rozliczanie zakładów. Silnik domenowy pozostaje niezależny od biblioteki Gymnasium. Kodowanie obserwacji, odwzorowanie akcji i konstrukcja nagrody pozostają poza zakresem warstwy realizującej zasady gry, co umożliwia porównywanie różnych reprezentacji stanu i funkcji nagrody bez modyfikowania samych reguł.

3.1. Założenia projektowe środowiska

Podstawową jednostką interakcji ze środowiskiem jest jedno kompletne rozdanie. Rozdanie rozpoczyna się od utworzenia talii i przekazania graczowi oraz krupierowi po dwóch kart własnych, a kończy się spasowaniem gracza albo automatycznym rozstrzygnięciem układów. Z punktu widzenia uczenia przez wzmocnianie pojedyncze rozdanie stanowi jeden epizod.

Agent podejmuje decyzje wyłącznie w trzech etapach: przed flopem, po odkryciu flopa oraz po odkryciu wszystkich pięciu kart wspólnych. W zależności od etapu może czekać, wykonać zakład Play o odpowiednim mnożniku albo spasować. Po wykonaniu zakładu Play środowisko automatycznie odkrywa pozostałe karty, ocenia ręce gracza i krupiera oraz przechodzi do stanu końcowego. W ramach jednego rozdania zakład Play może zostać wykonany tylko raz.

Środowisko modeluje podstawowy wariant Ultimate Texas Hold'em obejmujący zakłady Ante, Blind oraz Play. Zakłady boczne Trips i Progressive zostały pominięte. Nie wpływają one na przestrzeń decyzji podstawowej gry, natomiast zmieniają rozkład wyników finansowych i utrudniają ocenę jakości strategii decyzyjnej agenta. Uzasadnienie tego ograniczenia przedstawiono szerzej w dalszej części rozdziału.

Wartości zakładów wyrażono w jednostkach względnych, gdzie jedna jednostka odpowiada wartości zakładu Ante. Zakłady Ante i Blind mają zatem wartość jednej

jednostki, natomiast wartość zakładu Play zależy od momentu jego wykonania i może wynosić od jednej do czterech jednostek. Takie rozwiązanie uniezależnia środowisko od konkretnej waluty i nominalnej wysokości stawki, a także ułatwia porównywanie wyników pomiędzy eksperymentami.

Silnik zwraca szczegółowe rozliczenie poszczególnych zakładów oraz łączny wynik netto rozdania. Wynik netto stanowi podstawę nagrody terminalnej, a krokom pośrednim przypisano nagrodę zero. Rozdzielenie mechanizmu rozliczania gry od funkcji nagrody pozwala badać alternatywne warianty nagrody bez ingerencji w reguły Ultimate Texas Hold'em.

Istotnym założeniem była również powtarzalność eksperymentów. Standardowa talia może być tasowana przy użyciu zadanego ziarna generatora liczb pseudolosowych, dzięki czemu możliwe jest odtworzenie całej sekwencji rozdań. W testach jednostkowych stosowana jest dodatkowo talia deterministyczna, pozwalająca jednoznacznie określić kolejność dobieranych kart i zweryfikować konkretne przypadki przebiegu oraz rozliczenia rozdania.

3.2. Architektura rozwiązania

Środowisko Ultimate Texas Hold'em zaprojektowano jako zestaw niezależnych modułów domenowych. Każdy moduł odpowiada za jeden wyraźnie określony obszar, taki jak reprezentacja kart, ocena układów, sterowanie przebiegiem rozdania albo rozliczanie zakładów. Centralnym elementem rozwiązania jest klasa `UTHGame`, która koordynuje działanie pozostałych komponentów, ale nie implementuje bezpośrednio wszystkich reguł gry.

Przyjęty podział ogranicza zależności pomiędzy poszczególnymi częściami systemu. Model kart i evaluator układów mogą być rozwijane oraz testowane niezależnie od zasad Ultimate Texas Hold'em. Analogicznie mechanizm rozliczania zakładów nie odpowiada za odkrywanie kart ani za walidację dozwolonych akcji. Dzięki temu zmiana jednego elementu, na przykład reprezentacji obserwacji agenta, nie wymaga modyfikowania sposobu oceny układów lub obliczania wypłat.

Najniższą warstwę rozwiązania stanowią moduły `cards` i `hands`. Pierwszy z nich definiuje karty oraz talię, natomiast drugi odpowiada za ocenę i porównywanie układów pokerowych. Moduły te nie zawierają reguł charakterystycznych dla Ultimate Texas Hold'em i mogą zostać ponownie wykorzystane w innych odmianach pokera.

Warstwę domenową gry tworzy pakiet `uth`. Zawiera on modele stanu, reguły legalności akcji, mechanizm rozdawania kart, sposób rozliczania zakładów oraz logikę sterującą kolejnymi fazami rozdania. Klasa `UTHGame` pełni rolę koordynatora tej warstwy. Na podstawie aktualnego stanu i wybranej akcji wywołuje odpowiednie operacje, aktualizuje stan rozdania oraz zwraca rezultat kroku.

Agent nie komunikuje się bezpośrednio z talią, evaluatorem ani pełnym stanem

wewnętrznym gry. Otrzymuje obiekt `UTHObservation`, a swoją decyzję przekazuje do metody `step`. Rezultatem wykonania akcji jest obiekt `StepResult`, zawierający kolejną obserwację oraz, w przypadku zakończenia rozdania, jego wynik i szczegółowe rozliczenie. Takie rozwiązanie tworzy jednoznaczną granicę pomiędzy logiką środowiska a implementacją agenta.

Tabela 3.1 przedstawia najważniejsze moduły rozwiązania i zakres ich odpowiedzialności.

Tabela 3.1. Podział odpowiedzialności pomiędzy modułami środowiska UTH

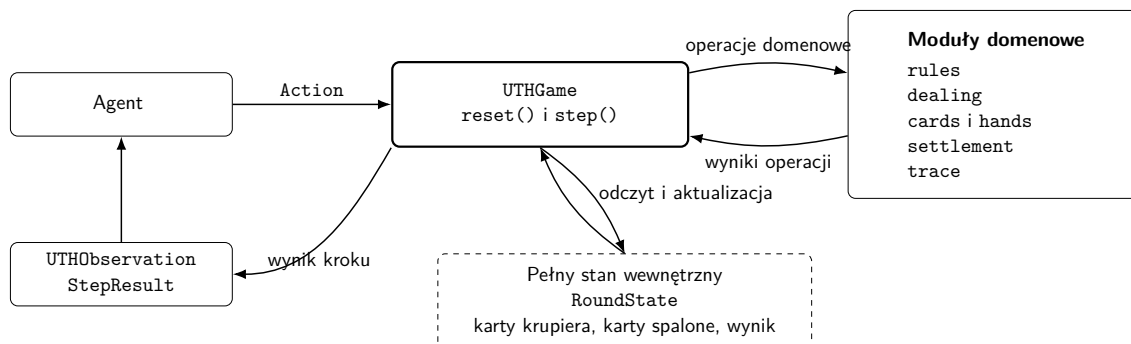
Moduł	Odpowiedzialność
<code>cards</code>	Reprezentacja rang, kolorów, pojedynczych kart oraz standardowej talii składającej się z 52 unikalnych kart.
<code>hands</code>	Ocena pięciokartowych układów, wybór najlepszego układu z pięciu, sześciu lub siedmiu kart oraz porównywanie wartości rąk.
<code>uth.models</code>	Definicja pełnego stanu rozdania, obserwacji agenta oraz wyniku pojedynczego kroku środowiska.
<code>uth.rules</code>	Określanie zbioru legalnych akcji dla każdej fazy rozdania.
<code>uth.dealing</code>	Realizacja kolejności rozdawania kart, odkrywania flopa, turna i rivera oraz obsługa kart spalonych.
<code>uth.card_source</code>	Abstrakcja źródła kart oraz deterministyczna talia wykorzystywana w testach.
<code>uth.settlement</code>	Określanie wyniku rozdania, kwalifikacji krupiera oraz rozliczanie zakładów Ante, Blind i Play.
<code>uth.game</code>	Koordinacja przebiegu rozdania, walidacja akcji, przechodzenie pomiędzy fazami oraz tworzenie wyników kolejnych kroków.
<code>uth.trace</code>	Opcjonalne rejestrowanie obserwacji, decyzji agenta i końcowego stanu rozdania na potrzeby diagnostyki.

Silnik domenowy nie zależy od konkretnej biblioteki uczenia przez wzmocnianie. Integrację z takimi bibliotekami wydzielono do osobnej warstwy korzystającej z publicznego interfejsu `UTHGame` i przekształcającej obiekty domenowe na reprezentacje numeryczne. Pozwala to zachować jedną implementację zasad gry dla wszystkich badanych reprezentacji obserwacji, funkcji nagrody i metod uczenia.

Ogólny przepływ informacji pomiędzy agentem, klasą sterującą i pozostałymi elementami silnika przedstawiono na rysunku 3.1. Agent komunikuje się wyłącznie z publicznym interfejsem klasy `UTHGame`. Pełny stan rozdania oraz moduły realizujące reguły gry pozostają ukryte za tą warstwą.

3.2.1. Przepływ danych podczas wykonania akcji

Metoda `reset()` zwraca pierwszą obserwację, na podstawie której agent wybiera akcję ze zbioru legalnego dla bieżącej fazy i przekazuje ją do metody `step()`. Silnik najpierw sprawdza, czy rozdanie nie zostało zakończone oraz czy akcja jest poprawna i legalna. Odrzucenie następuje przed jakąkolwiek zmianą stanu i pobraniem kart, co zapobiega częściowemu wykonaniu niedozwolonego przejścia. Po pozytywnej walidacji wykonywana jest operacja właściwa dla fazy, prowadząca do odkrycia kolejnych kart



Rys. 3.1. Architektura i przepływ informacji w środowisku UTH

albo do showdownu i rozliczenia.

Wynikiem przejścia jest nowy, niemodyfikowalny obiekt `RoundState`, na podstawie którego budowane są obserwacja agenta oraz obiekt `StepResult` z informacją o zakończeniu rozdania i, w stanie terminalnym, o jego wyniku i rozliczeniu. Jeżeli rejestrowanie historii jest włączone, obserwacja sprzed decyzji i wybrana akcja są zapisywane dopiero po udanym przejściu, dzięki czemu historia pozostaje spójna z rzeczywistością wykonanymi krokami.

3.3. Model kart i sterowanie talią

Reprezentacja kart i mechanizm sterowania talią stanowią podstawową warstwę środowiska. Elementy te zostały zaprojektowane niezależnie od zasad Ultimate Texas Hold'em, dzięki czemu mogą być wykorzystywane także przez inne warianty pokera oraz przez moduł oceniający układy.

3.3.1. Reprezentacja karty

Pojedyncza karta jest reprezentowana przez niemodyfikowalny obiekt `Card`, zawierający dwa pola: rangę oraz kolor. Rangi zapisano za pomocą typu wyliczeniowego `Rank`, natomiast kolory za pomocą typu `Suit`. Zastosowanie typów wyliczeniowych ogranicza możliwość utworzenia karty zawierającej niepoprawną wartość.

Rangi kart przyjmują wartości liczbowe od 2 do 14. Karty od dwójki do dziesiątki zachowują swoje naturalne wartości, natomiast walet, dama, król i as otrzymują odpowiednio wartości 11, 12, 13 i 14. Przyjęta reprezentacja umożliwia bezpośrednie porównywanie rang podczas oceny układów pokerowych. W podstawowej reprezentacji as jest kartą najwyższą, natomiast szczególny przypadek strita od asa do piątki obsługiwany jest przez evaluator układów.

Zbiór kolorów obejmuje trefle, karo, kiery i piki.

Obiekty kart są niemodyfikowalne po ich utworzeniu. Pozwala to bezpiecznie umieszczać je w krotkach i zbiorach, a także wykorzystywać porównanie liczności zbioru do wykrywania zduplikowanych kart. Konstruktor dodatkowo sprawdza, czy przekazana ranga i kolor należą do odpowiednich typów wyliczeniowych.

3.3.2. Standardowa talia

Standardową talię reprezentuje klasa `Deck`. Podczas jej tworzenia generowane są wszystkie kombinacje trzynastu rang i czterech kolorów. Liczba kart w pełnej talii wynosi zatem

$$|\mathcal{D}| = 13 \cdot 4 = 52. \quad (3.1)$$

Ponieważ każda kombinacja rangi i koloru występuje dokładnie raz, wygenerowana talia zawiera 52 unikalne karty.

Podstawową operacją jest metoda `draw()`, która pobiera jedną kartę z wierzchu talii i jednocześnie usuwa ją z kolekcji. Dostępna jest również operacja pobrania wielu kart. Próba pobrania karty z pustej talii lub większej liczby kart niż liczba pozostałych elementów powoduje zgłoszenie wyjątku. Takie zachowanie umożliwia szybkie wykrycie niepoprawnej implementacji przebiegu rozdania.

Domyślnie talia jest tasowana bezpośrednio po utworzeniu. Możliwe jest także utworzenie talii w kolejności początkowej, co bywa użyteczne podczas testowania samego modelu kart. Tasowanie wykonywane jest przy użyciu lokalnego generatora liczb pseudolosowych, a nie globalnego stanu losowości programu.

3.3.3. Abstrakcja źródła kart

Silnik rozdania nie zależy bezpośrednio od klasy `Deck`. Zamiast tego korzysta z abstrakcji `CardSource`, która wymaga jedynie udostępnienia operacji `draw()`. Każdy obiekt spełniający ten kontrakt może zostać użyty jako źródło kolejnych kart.

Minimalny interfejs źródła kart oddziela logikę rozdania od sposobu przechowywania i wybierania kart. W normalnej rozgrywce źródłem jest pseudolosowo przetasowana talia, natomiast w testach można przekazać źródło deterministyczne. Moduł odpowiedzialny za rozdawanie wykonuje w obu przypadkach te same operacje i nie musi rozpoznawać rodzaju używanej talii.

3.3.4. Deterministyczna talia testowa

Do testowania konkretnych scenariuszy wprowadzono klasę `FixedDeck`. Podczas jej tworzenia przekazywana jest jawna sekwencja kart określająca kolejność dobierania. Pierwszy element przekazanej sekwencji jest pierwszą kartą zwracaną przez metodę `draw()`.

Przed zapisaniem sekwencji sprawdzane jest, czy wszystkie jej elementy są poprawnymi obiektami `Card` oraz czy żadna karta nie występuje więcej niż raz. Dzięki temu błąd w danych testowych nie może doprowadzić do przeprowadzenia rozdania zawierającego dwie identyczne karty.

Deterministyczne źródło umożliwia przygotowanie powtarzalnych testów obejmu-

jących ściśle określone scenariusze rozgrywki, kwalifikacji krupiera i rozliczenia zakładów.

3.3.5. Powtarzalność losowych rozdań

Powtarzalność eksperymentów zapewniono przez możliwość przekazania ziarna generatora liczb pseudolosowych. Utworzenie dwóch talii z tym samym ziarnem prowadzi do uzyskania tej samej kolejności kart, o ile wykonano na nich tę samą sekwencję operacji.

Na poziomie klasy UTHGame zastosowano dodatkowy generator nadrzędny. Przy każdym rozpoczęciu nowego rozdania generuje on osobne ziarno przekazywane do nowej talii. W rezultacie kolejne rozdania w obrębie jednego eksperymentu różnią się od siebie, ale cała ich sekwencja może zostać odtworzona przez ponowne uruchomienie środowiska z tym samym ziarnem początkowym.

Rozwiązanie to pozwala porównywać agentów na odpowiadających sobie sekwencjach rozdań. Jeżeli dwa eksperymenty zostaną uruchomione z tym samym ziarnem środowiska i zachowają tę samą kolejność wywołań, otrzymają identyczne strumienie talii. Ogranicza to wpływ losowego doboru kart na ocenę różnic pomiędzy badanymi strategiami.

3.4. Ocena układów pokerowych

Moduł oceny układów pokerowych został zaimplementowany niezależnie od reguł Ultimate Texas Hold'em. Jego zadaniem jest określenie wartości dowolnego poprawnego układu pięciokartowego oraz wybranie najlepszej możliwej ręki z pięciu, sześciu lub siedmiu dostępnych kart. W środowisku UTH evaluator jest wykorzystywany podczas showdownu osobno dla gracza i krupiera.

Hierarchię układów pokerowych oraz ich znaczenie przedstawiono w rozdziale teoretycznym. W niniejszej sekcji opisano sposób odwzorowania tej hierarchii w implementacji oraz mechanizm jednoznacznego porównywania dwóch rąk.

3.4.1. Reprezentacja wartości ręki

Kategorię układu reprezentuje typ wyliczeniowy HandRank. Kolejnym kategoriom przypisano rosnące wartości liczbowe, począwszy od wysokiej karty, a kończąc na pokerze. Dzięki temu porównanie kategorii może zostać wykonane przez porównanie odpowiadających im wartości całkowitych.

Sama kategoria nie wystarcza jednak do rozstrzygnięcia wszystkich przypadków. Dwie ręce mogą należeć do tej samej kategorii, lecz różnić się rangą kart tworzących układ albo kart dodatkowych. Z tego względu pełna wartość ręki jest reprezentowana przez obiekt HandValue, zawierający kategorię układu rank oraz uporządkowaną krotkę wartości rozstrzygających remis tiebreak.

Klucz porównawczy ręki można zapisać jako

$$K(h) = (r(h), t_1(h), t_2(h), \dots, t_n(h)), \quad (3.2)$$

gdzie $r(h)$ oznacza wartość kategorii układu, natomiast $t_1(h), \dots, t_n(h)$ są kolejnymi wartościami rozstrzygającymi remis. Dwa klucze porównywane są leksykograficznie. W pierwszej kolejności porównywana jest kategoria układu, a dopiero w przypadku jej równości kolejne elementy krotki tiebreak.

Przykładowo wartość pary asów z królem, dziesiątką i dziewiątką jako kartami dodatkowymi może zostać zapisana w postaci

$$K(h) = (\text{ONE_PAIR}, 14, 13, 10, 9). \quad (3.3)$$

Jeżeli druga ręka również zawiera parę asów, porównanie przechodzi kolejno do króla, dziesiątki i dziewiątki. Kolory kart nie uczestniczą w rozstrzyganiu remisów, zgodnie ze standardowymi zasadami pokera.

Obiekt `HandValue` sprawdza zgodność długości krotki tiebreak z kategorią układu. Walidowane jest również, czy wszystkie jej elementy są liczbami całkowitymi odpowiadającymi poprawnym rangom kart. Ogranicza to możliwość utworzenia wartości ręki, która nie mogłaby powstać w prawidłowym rozdaniu.

3.4.2. Ocena układu pięciokartowego

Funkcja `evaluate_five_card_hand()` przyjmuje dokładnie pięć unikalnych kart. Przed rozpoczęciem oceny sprawdzana jest liczba kart, typ każdego elementu oraz brak duplikatów.

W pierwszym kroku obliczana jest liczba wystąpień każdej rangi. Na tej podstawie możliwe jest wykrycie par, dwóch par, trójki, fulla oraz karety. Niezależnie sprawdzane są dwa warunki: czy wszystkie karty mają ten sam kolor oraz czy pięć różnych rang tworzy ciąg kolejnych wartości.

Kategorie sprawdzane są od najsilniejszej do najsłabszej. Jeżeli spełnione są jednocześnie warunki strita i koloru, zwracany jest poker. W przeciwnym razie sprawdzane są kolejno kareta, full, kolor, strit, trójka, dwie pary, jedna para oraz wysoka karta.

Wartość tiebreak jest budowana bezpośrednio podczas rozpoznawania kategorii. Wynikiem funkcji nie jest zatem jedynie nazwa układu, lecz kompletna wartość umożliwiająca porównanie z dowolną inną poprawną ręką. Szczególnym przypadkiem jest strit *A-2-3-4-5*, w którym as pełni rolę najniższej karty, evaluator zwraca wtedy strita o najwyższej karcie równej 5, bez zmiany wartości asa w pozostałych kategoriach.

3.4.3. Wybór najlepszej ręki

Podczas showdownu gracz i krupier dysponują łącznie siedmioma kartami: dwiema kartami własnymi oraz pięcioma kartami wspólnymi. Rękę pokerową tworzy jednak do-

kładnie pięć kart. Funkcja `evaluate_best_hand()` generuje wszystkie możliwe pięciokartowe podzbiory dostępnych kart, ocenia każdy z nich, a następnie wybiera układ o największej wartości `HandValue`.

Liczba sprawdzanych kombinacji dla n dostępnych kart wynosi

$$N(n) = \binom{n}{5}. \quad (3.4)$$

Dla liczby kart obsługiwanych przez evaluator otrzymuje się:

$$\binom{5}{5} = 1, \quad \binom{6}{5} = 6, \quad \binom{7}{5} = 21. \quad (3.5)$$

W przypadku standardowego showdownu UTH konieczne jest zatem sprawdzenie 21 kombinacji dla gracza oraz 21 kombinacji dla krupiera. Ze względu na niewielką i stałą liczbę możliwości pełne przeszukanie jest wystarczająco proste, jednoznaczne oraz łatwe do zweryfikowania w testach. Nie było potrzeby stosowania bardziej złożonego, specjalizowanego algorytmu ewaluacji.

Przed wygenerowaniem kombinacji sprawdzane jest, czy przekazano od pięciu do siedmiu kart, czy wszystkie elementy są kartami oraz czy żadna karta nie występuje więcej niż raz.

Wynikiem operacji jest obiekt `EvaluatedHand`, który przechowuje zarówno wartość `HandValue`, jak i dokładnie pięć kart tworzących wybrany układ, co jest przydatne przy prezentowaniu wyniku showdownu oraz diagnostyce evaluatora.

Poker królewski nie stanowi osobnej kategorii `HandRank`. Jest reprezentowany jako poker z asem jako najwyższą kartą. Obiekt `HandValue` udostępnia właściwość pozwalającą rozpoznać ten przypadek, co jest potrzebne przy naliczaniu najwyższej wypłaty zakładu `Blind`.

3.5. Stan wewnętrzny i obserwacja agenta

Ultimate Texas Hold'em jest grą z niepełną informacją. W momencie podejmowania decyzji gracz zna własne karty oraz dotychczas odkryte karty wspólne, ale nie zna kart krupiera, kart spalonych ani kolejności pozostałych kart w talii. Poprawne odwzorowanie tej własności wymaga rozdzielenia pełnego stanu środowiska od informacji udostępnianych agentowi.

W implementacji zastosowano dwa odrębne modele danych: `RoundState` reprezentujący pełny stan wewnętrzny rozdania oraz `UTHObservation` reprezentujący obserwację dostępną agentowi.

Agent nie otrzymuje bezpośredniego odwołania do obiektu `RoundState`. Każda obserwacja jest tworzona na jego podstawie przez osobną funkcję projekcji, która kopiuje wyłącznie informacje dozwolone z perspektywy gracza.

3.5.1. Pełny stan rozdania

Obiekt `RoundState` zawiera wszystkie dane potrzebne do kontynuowania i rozstrzygnięcia rozdania. Obejmuje on:

- aktualną fazę gry,
- dwie karty własne gracza,
- dwie ukryte karty krupiera,
- odkryte karty wspólne,
- karty spalone,
- mnożnik wykonanego zakładu `Play`,
- końcowy wynik rozdania,
- szczegółowe rozliczenie zakładów,
- rezultat showdownu.

Nie wszystkie pola są aktywne jednocześnie. W stanach pośrednich wynik rozdania, rozliczenie, rezultat showdownu oraz mnożnik zakładu `Play` pozostają nieokreślone. Pola te otrzymują wartości dopiero po przejściu do stanu terminalnego.

Liczba kart wspólnych i spalonych jest uzależniona od aktualnej fazy. Zależność tę przedstawiono w tabeli 3.2.

Tabela 3.2. Liczba kart przechowywanych w stanie w poszczególnych fazach

Faza	Karty wspólne	Karty spalone	Znaczenie
PREFLOP	0	0	Rozdano wyłącznie karty własne gracza i krupiera.
FLOP	3	1	Spalono jedną kartę i odkryto flop.
RIVER	5	2	Odkryto komplet kart wspólnych.
TERMINAL	5	2	Rozdanie zostało zakończone fol-dem albo showdownem.

Obiekt stanu jest niemodyfikowalny. Każde przejście środowiska prowadzi do utworzenia nowej instancji `RoundState`, zamiast modyfikowania istniejącego obiektu. Upraszcza to analizę przejść, ogranicza ryzyko częściowej aktualizacji oraz ułatwia tworzenie deterministycznych testów.

Podczas tworzenia stanu sprawdzane są między innymi:

- poprawny typ fazy,
- obecność dokładnie dwóch kart gracza i dwóch kart krupiera,

- liczba kart wspólnych właściwa dla danej fazy,
- liczba kart spalonych właściwa dla danej fazy,
- brak zduplikowanych kart w całym stanie,
- zgodność pól końcowych z fazą rozdania.

Walidacja obejmuje również zależności semantyczne pomiędzy polami. Stan niekońcowy nie może zawierać wyniku ani rozliczenia. Stan zakończony foldem nie może zawierać mnożnika Play ani rezultatu showdownu. Z kolei stan zakończony showdownem musi zawierać zarówno mnożnik wykonanego zakładu, jak i szczegóły ocenionych rąk.

3.5.2. Obserwacja agenta

Obiekt `UTHObservation` zawiera wyłącznie dane, które mogą zostać wykorzystane przez agenta podczas podejmowania decyzji. Są to:

- aktualna faza gry,
- dwie karty własne gracza,
- dotychczas odkryte karty wspólne,
- zbiór legalnych akcji,
- mnożnik zakładu Play w obserwacji terminalnej.

Obserwacja celowo nie zawiera:

- kart krupiera,
- kart spalonych,
- pozostałych kart w talii,
- kolejności przyszłego dobierania kart,
- wyniku ewaluacji ręki krupiera przed zakończeniem rozdania.

Zbiór legalnych akcji jest umieszczony bezpośrednio w obserwacji. Agent nie musi zatem samodzielnie odtwarzać zasad gry na podstawie fazy. Rozwiązanie to ułatwia implementację agentów bazowych i pozwala warstwie integracyjnej zbudować maskę akcji bez sięgania do wewnętrznego stanu.

Obserwacja, podobnie jak pełny stan, jest obiektem niemodyfikowalnym. Sprawdzana jest liczba widocznych kart, brak duplikatów oraz zgodność zbioru legalnych akcji z aktualną fazą gry. Dzięki temu błędnie skonstruowana obserwacja jest odrzucana przed przekazaniem jej agentowi.

3.5.3. Projekcja stanu na obserwację

Proces tworzenia obserwacji można przedstawić jako funkcję projekcji

$$O_t = \phi(S_t), \quad (3.6)$$

gdzie S_t jest pełnym stanem środowiska w chwili t , natomiast O_t oznacza obserwację udostępnioną agentowi.

Dla zaimplementowanego środowiska projekcja ma postać

$$\phi(S_t) = (p_t, C_t^{player}, C_t^{community}, A_t^{legal}, m_t), \quad (3.7)$$

gdzie:

- p_t oznacza aktualną fazę gry,
- C_t^{player} oznacza karty własne gracza,
- $C_t^{community}$ oznacza odkryte karty wspólne,
- A_t^{legal} oznacza zbiór legalnych akcji,
- m_t oznacza mnożnik zakładu Play, jeżeli został już wykonany.

Karty krupiera i karty spalone należą do S_t , natomiast kolejność pozostałej talii jest przechowywana przez wewnętrzne źródło kart. Żadna z tych informacji nie jest elementem O_t .

Porównanie zawartości pełnego stanu i obserwacji przedstawiono w tabeli 3.3.

Tabela 3.3. Porównanie pełnego stanu środowiska i obserwacji agenta

Informacja	RoundState	UTHObservation
Aktualna faza	tak	tak
Karty własne gracza	tak	tak
Odkryte karty wspólne	tak	tak
Karty krupiera	tak	nie
Karty spalone	tak	nie
Legalne akcje	wyznaczane na podstawie fazy	tak
Mnożnik zakładu Play	tak	tylko po wykonaniu zakładu
Końcowy wynik rozdania	tak	nie
Szczegółowe rozliczenie	tak	nie
Szczegóły showdownu	tak	nie

Wynik rozdania, rozliczenie i szczegóły showdownu są po zakończeniu epizodu zwracane przez obiekt `StepResult`, a nie przez samą obserwację. Informacje te pojawiają się dopiero po wykonaniu ostatniej decyzji, dlatego nie mogą wpłynąć na wybór akcji w bieżącym rozdaniu.

3.5.4. Ochrona przed wyciekami informacji

Rozdzielenie stanu i obserwacji pełni rolę ochrony przed wyciekami informacji ukrytej. Funkcja tworząca obserwację nie usuwa niepożądanych pól z kopii pełnego stanu, lecz jawnie konstruuje nowy obiekt wyłącznie z dozwolonych wartości. Takie podejście oparte na liście pól dozwolonych jest bezpieczniejsze niż lista pól zabronionych: nowe pole RoundState nie zostaje ujawnione agentowi, dopóki nie doda się go jawnie do modelu UTHObservation i do funkcji projekcji.

Informacje ukryte są częścią stanu potrzebnego do prowadzenia rozgrywki, lecz nie uczestniczą w tworzeniu obserwacji. Silnik musi przechowywać karty krupiera, aby przeprowadzić showdown, a karty spalone i kolejność talii są potrzebne do deterministycznego odtworzenia rozdania. Ich ujawnienie zmieniałoby jednak problem decyzyjny, dając agentowi wiedzę o przyszłych zdarzeniach losowych. Z tego powodu pełny stan jest wykorzystywany przez wewnętrzne komponenty silnika, natomiast poza nimi udostępnia się go wyłącznie na potrzeby testów, rejestrowania przebiegu i analizy zakończonych epizodów. Nie jest on przekazywany metodzie wybierającej akcję agenta.

Jawna funkcja projekcji stanowi zatem część definicji środowiska z niepełną informacją i zapewnia, że każda metoda wyboru akcji otrzymuje ten sam zakres informacji. Sposób numerycznego kodowania obserwacji pozostaje przy tym poza warstwą domenową, natomiast zbiór informacji dostępnych agentowi jest stały.

3.6. Maszyna stanów i przestrzeń akcji

Przebieg pojedynczego rozdania Ultimate Texas Hold'em odwzorowano jako maszynę stanów. Następna faza jest jednoznacznie wyznaczona przez bieżącą fazę i akcję, natomiast konkretna zawartość kolejnego stanu zależy od kart zwróconych przez źródło. Aktualna faza określa zakres informacji widocznych dla agenta, zbiór legalnych akcji oraz operacje wykonywane przez środowisko po otrzymaniu decyzji. Zbiór faz ma postać

$$\mathcal{P} = \{\text{PREFLOP}, \text{FLOP}, \text{RIVER}, \text{TERMINAL}\}. \quad (3.8)$$

Rozdanie rozpoczyna metoda reset(), która rozdaje po dwie karty graczowi i krupierowi w kolejności P_1, D_1, P_2, D_2 , tworzy stan PREFLOP i zwraca pierwszą obserwację. Karty wspólne i spalone nie są jeszcze dobierane. Każde poprawne wykonanie metody step() przesuwa rozdanie do kolejnej fazy albo do stanu terminalnego. Agent nie może pozostać w tej samej fazie po wykonaniu akcji.

Pełna przestrzeń akcji zawiera sześć decyzji domenowych, a zbiór akcji legalnych zależy od fazy:

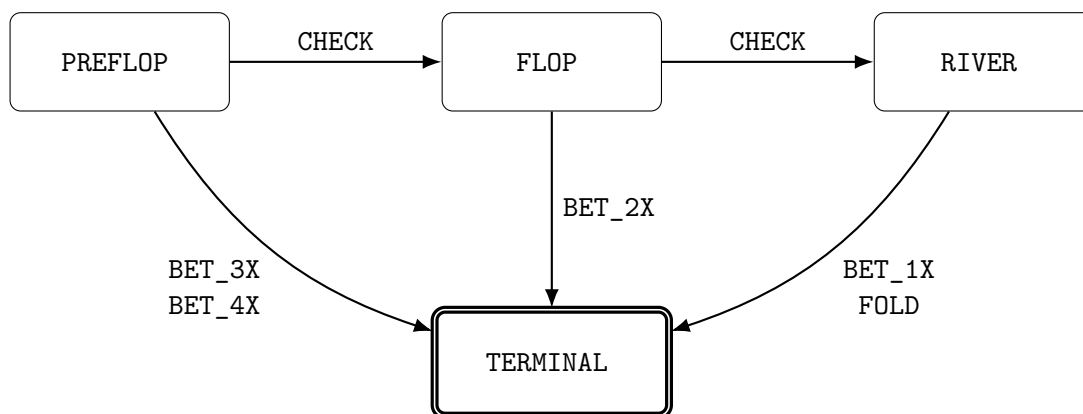
$$\mathcal{A} = \{\text{CHECK}, \text{BET_4X}, \text{BET_3X}, \text{BET_2X}, \text{BET_1X}, \text{FOLD}\}, \quad (3.9)$$

$$\mathcal{A}_{legal}(p) = \begin{cases} \{\text{CHECK}, \text{BET_3X}, \text{BET_4X}\}, & p = \text{PREFLOP}, \\ \{\text{CHECK}, \text{BET_2X}\}, & p = \text{FLOP}, \\ \{\text{BET_1X}, \text{FOLD}\}, & p = \text{RIVER}, \\ \emptyset, & p = \text{TERMINAL}. \end{cases} \quad (3.10)$$

Akcja CHECK odkłada decyzję o zakładzie Play i przesuwa rozdanie do następnej fazy, spalając kartę oraz odkrywając flop albo turn i river. Każda akcja typu BET ustala mnożnik Play wynikający z fazy: 4× lub 3× przed flopem, 2× po flopie i 1× na riverze. Po zakładzie środowisko odkrywa pozostałe karty wspólne, przeprowadza showdown i przechodzi do stanu terminalnego. Akcja FOLD kończy rozdanie na riverze bez showdownu. Zbiór legalnych akcji w poszczególnych fazach zestawiono w tabeli 3.4, a przejścia przedstawiono na rysunku 3.2.

Tabela 3.4. Legalne akcje w poszczególnych fazach środowiska UTH

Faza	Legalne akcje
PREFLOP	CHECK, BET_3X, BET_4X
FLOP	CHECK, BET_2X
RIVER	BET_1X, FOLD
TERMINAL	brak



Rys. 3.2. Maszyna stanów pojedynczego rozdania UTH

3.6.1. Pojedynczy zakład Play i walidacja decyzji

Struktura maszyny stanów gwarantuje, że zakład Play może zostać wykonany najwyżej raz: każda akcja typu BET prowadzi bezpośrednio do stanu terminalnego, a akcje CHECK jedynie przesuwały decyzję do późniejszej fazy. Najdłuższa ścieżka zawiera trzy decyzje (CHECK, CHECK, a następnie BET_1X albo FOLD), a najkrótsza jedną, po zakładzie 3× lub 4× przed flopem. Zastosowanie osobnych akcji BET_1X, BET_2X, BET_3X i BET_4X zamiast jednej ogólnej akcji BET wiąże mnożnik Play jednoznacznie z fazą i eliminuje możliwość przekazania niepoprawnej wysokości zakładu.

Zbiór legalnych akcji jest wyznaczany przez środowisko i umieszczany w obiekcie `UTHObservation`, dzięki czemu agent nie musi samodzielnie odtwarzać reguł legalności. Nie oznacza to jednak, że środowisko ufa zwróconej decyzji - każda akcja jest ponownie walidowana przez metodę `step()`. Przed wykonaniem operacji zmieniającej stan sprawdzane jest kolejno, czy rozdanie zostało rozpoczęte, czy stan nie jest terminalny, czy przekazana wartość jest obiektem typu `Action` oraz czy należy do zbioru $\mathcal{A}_{legal}$ dla bieżącej fazy. Naruszenie tych warunków powoduje zgłoszenie odpowiednio `RoundNotStartedError`, `RoundFinishedError` albo `IllegalActionError`.

Walidacja następuje przed dobieraniem kart i zapisaniem nowego stanu, więc nielegalna decyzja nie zużywa kart z talii, nie zmienia fazy rozdania i nie jest dodawana do historii. Środowisko nie zastępuje nielegalnej akcji inną decyzją ani nie nakłada automatycznej kary, a obsługa błędów należy do komponentu sterującego eksperymentem.

3.7. Showdown i rozliczanie zakładów

Każde rozdanie, w którym gracz wykonał zakład `Play`, kończy się showdownem. Środowisko odkrywa wszystkie brakujące karty wspólne, ocenia najlepszą pięciokartową rękę gracza i krupiera, a następnie porównuje otrzymane wartości. Wynik porównania stanowi podstawę do rozliczenia zakładów `Ante`, `Blind` oraz `Play`.

Spasowanie gracza jest obsługiwane oddzielnie. W takim przypadku showdown nie jest przeprowadzany, ponieważ wynik rozdania wynika bezpośrednio z decyzji gracza.

3.7.1. Przebieg showdownu

Podczas showdownu każda strona dysponuje siedmioma kartami:

$$C^{player} = C_{private}^{player} \cup C^{community}, \quad (3.11)$$

$$C^{dealer} = C_{private}^{dealer} \cup C^{community}, \quad (3.12)$$

gdzie $C_{private}^{player}$ i $C_{private}^{dealer}$ oznaczają po dwie karty własne, natomiast $C^{community}$ zawiera pięć kart wspólnych.

Evaluator wybiera najlepszy pięciokartowy układ osobno dla gracza i krupiera. Wynik tej operacji jest przechowywany w obiekcie `ShowdownResult`, który zawiera:

- najlepszą rękę gracza,
- najlepszą rękę krupiera,
- wynik porównania z perspektywy gracza,
- informację o kwalifikacji krupiera.

Wynik showdownu należy do zbioru

$$O_{\text{showdown}} = \{\text{PLAYER_WIN}, \text{DEALER_WIN}, \text{PUSH}\}. \quad (3.13)$$

Dla wartości rąk H_p oraz H_d wynik można zdefiniować jako

$$O(H_p, H_d) = \begin{cases} \text{PLAYER_WIN}, & H_p > H_d, \\ \text{DEALER_WIN}, & H_p < H_d, \\ \text{PUSH}, & H_p = H_d. \end{cases} \quad (3.14)$$

Porównanie rąk jest wykonywane niezależnie od kwalifikacji krupiera. Oznacza to, że krupier może wygrać rozdanie również wtedy, gdy jego ręka nie spełnia warunku kwalifikacji. Przykładem jest sytuacja, w której obie strony mają wyłącznie wysoką kartę, ale wysoka karta krupiera jest silniejsza.

3.7.2. Kwalifikacja krupiera

W podstawowym wariantcie Ultimate Texas Hold'em krupier kwalifikuje się, jeżeli posiada co najmniej jedną parę. Warunek kwalifikacji można zapisać jako

$$Q(H_d) = \begin{cases} 1, & \text{rank}(H_d) \geq \text{ONE_PAIR}, \\ 0, & \text{rank}(H_d) = \text{HIGH_CARD}. \end{cases} \quad (3.15)$$

Kwalifikacja nie zmienia wyniku porównania rąk. Wpływa natomiast na sposób rozliczenia zakładu Ante:

- jeżeli krupier kwalifikuje się, Ante wygrywa lub przegrywa zgodnie z wynikiem showdownu,
- jeżeli krupier nie kwalifikuje się, Ante jest zwracane niezależnie od tego, która strona ma silniejszą rękę.

Zakład Play jest zawsze rozliczany na podstawie bezpośredniego porównania rąk. Zakład Blind jest rozliczany według układu gracza w przypadku jego zwycięstwa, zwracany przy remisie oraz przegrywany przy zwycięstwie krupiera.

Zależności te przedstawiono w tabeli 3.5.

Tabela 3.5. Rozliczenie zakładów po showdownie

Wynik	Kwalifikacja	Ante	Blind	Play
Wygrana gracza	tak	wygrana 1:1	według tabeli wypłat	wygrana 1:1
Wygrana gracza	nie	zwrot	według tabeli wypłat	wygrana 1:1
Remis	dowolna	zwrot	zwrot	zwrot
Wygrana krupiera	tak	przegrana	przegrana	przegrana
Wygrana krupiera	nie	zwrot	przegrana	przegrana

Ostatni przypadek jest istotny z punktu widzenia poprawności implementacji. Brak kwalifikacji krupiera nie oznacza automatycznego zwycięstwa gracza. Jeżeli krupier ma silniejszą wysoką kartę, wygrywa porównanie, zakłady Blind i Play są przegrywane, natomiast Ante zostaje zwrócone.

3.7.3. Model rozliczenia zakładów

Wynik finansowy każdego zakładu reprezentuje obiekt `WagerSettlement`. Zawiera on dwie wartości:

- `stake` – początkową wartość postawionego zakładu,
- `net_profit` – zysk albo stratę netto.

Wszystkie kwoty są wyrażane w jednostkach Ante. Dla pojedynczego zakładu o stawce s i zysku netto n całkowity zwrot brutto wynosi $g = s + n$; wartość $g = s$ oznacza zwrot stawki, $g = 0$ jej pełną utratę, a stawka $s = 0$ zakład niepostawiony.

Pełne rozliczenie rozdania reprezentuje obiekt `Settlement`, zawierający osobne wyniki dla Ante, Blind oraz Play. Umożliwia to analizę zarówno łącznego wyniku rozdania, jak i wpływu poszczególnych składników. Łączny wynik netto jest sumą składników

$$S_{net} = n_{Ante} + n_{Blind} + n_{Play}. \quad (3.16)$$

Wartości finansowe są reprezentowane za pomocą typu `Fraction`. Pozwala to przechowywać wypłaty takie jak $3/2$ bez stosowania przybliżeń zmiennoprzecinkowych. Ma to znaczenie podczas wielokrotnego sumowania wyników w eksperymentach, ponieważ eliminuje błędy zaokrągleń na poziomie pojedynczych rozdań.

3.7.4. Rozliczenie zakładu Blind

Zakład Blind ma stałą stawkę równą jednej jednostce Ante. W przypadku zwycięstwa gracza jego zysk zależy od kategorii uzyskanego układu. Dla układów słabszych od strita Blind jest zwracany. Dla strita lub silniejszego układu stosowana jest tabela wypłat przedstawiona w tabeli 3.6.

Poker królewski jest w evaluatorze szczególnym przypadkiem pokera z asem jako najwyższą kartą. Nie stanowi osobnej kategorii układu, ale jest rozpoznawany podczas wyznaczania wypłaty Blind.

Tabela Blind jest stosowana wyłącznie wtedy, gdy gracz wygrał showdown. Przy remisie zakład jest zwracany, natomiast przy zwycięstwie krupiera pełna stawka Blind zostaje utracona.

3.7.5. Rozliczenie Ante i Play

Zakład Ante ma stałą wartość jednej jednostki. Przy remisie jest zwracany. Jeżeli krupier kwalifikuje się, Ante wygrywa albo przegrywa zgodnie z wynikiem showdownu.

Tabela 3.6. Tabela wypłat zakładu Blind

Układ gracza	Wypłata	Zysk netto
Wysoka karta	zwrot	0
Jedna para	zwrot	0
Dwie pary	zwrot	0
Trójka	zwrot	0
Strit	1:1	1
Kolor	3:2	$\frac{3}{2}$
Full	3:1	3
Kareta	10:1	10
Poker	50:1	50
Poker królewski	500:1	500

W przypadku braku kwalifikacji zakład jest zwracany.

Wartość zakładu Play jest określona przez moment podjęcia decyzji:

$$s_{Play} \in \{1, 2, 3, 4\}. \quad (3.17)$$

Zakład Play jest wypłacany 1:1, niezależnie od jego mnożnika. Przykładowo wygrany Play $4\times$ daje zysk netto równy czterem jednostkom. Przy porażce tracona jest pełna wartość zakładu, natomiast przy remisie cała stawka zostaje zwrócona.

3.7.6. Rozliczenie spasowania

Jeżeli gracz spasuje na riverze, nie dochodzi do showdownu. Ante i Blind zostają przegrane, natomiast Play nie został postawiony.

Rozliczenie folda ma zatem postać

$$S_{fold} = (n_{Ante} = -1, n_{Blind} = -1, n_{Play} = 0). \quad (3.18)$$

Łączna stawka wynosi dwie jednostki, a wynik netto jest równy

$$S_{net}^{fold} = -2. \quad (3.19)$$

W rozliczeniu Play zapisywana jest stawka równa zero. Pozwala to odróżnić zakład niepostawiony od postawionego zakładu, który został zwrócony.

3.7.7. Przykładowe rozliczenie

Nietrywialnym przypadkiem jest rozdanie, w którym niekwalifikujący się krupier ma wyższą wysoką kartę niż gracz. Przy zakładzie Play $1\times$ Ante zostaje zwrócone, natomiast Blind i Play są przegrane, co daje wynik netto

$$S_{net} = 0 - 1 - 1 = -2. \quad (3.20)$$

Przykład ten pokazuje, że warunek kwalifikacji wpływa jedynie na rozliczenie Ante i nie zastępuje porównania układów.

3.8. Funkcja nagrody i wynik epizodu

Mechanizm rozliczania zakładów został oddzielony od funkcji nagrody wykorzystywanej przez algorytmy uczenia przez wzmacnianie. Silnik domenowy wyznacza wyłącznie dokładny wynik finansowy rozdania zgodny z zasadami Ultimate Texas Hold'em, natomiast konstruowanie sygnału nagrody należy do warstwy integrującej silnik z agentem. Dzięki temu ta sama implementacja zasad gry może być wykorzystywana w eksperymentach z różnymi funkcjami nagrody, bez modyfikowania showdownu, kwalifikacji krupiera ani tabel wypłat.

Pojedyncze rozdanie stanowi jeden epizod. W podstawowym wariacie źródłem nagrody terminalnej jest łączny wynik netto rozdania $S_{net} = n_{Ante} + n_{Blind} + n_{Play}$, a wszystkie przejścia pośrednie otrzymują nagrodę równą zero:

$$r_t = \begin{cases} 0, & s_{t+1} \notin \mathcal{S}_{terminal}, \\ S_{net}, & s_{t+1} \in \mathcal{S}_{terminal}. \end{cases} \quad (3.21)$$

Wynik netto wybrano zamiast zwrotu brutto, ponieważ nie zawiera zwracanych stawek początkowych: zwrócony zakład otrzymuje wtedy wartość zero. Wartości rozliczeń są przechowywane za pomocą typu `Fraction`. Konwersję do reprezentacji numerycznej wymaganej przez algorytm uczący pozostawiono warstwie integracyjnej.

3.9. Interfejs silnika

Silnik domenowy udostępnia niewielki interfejs publiczny, za pomocą którego można rozpocząć rozdanie, pobrać obserwację, wykonać decyzję oraz odczytać wynik zakończonego epizodu. Interfejs nie zależy od konkretnego algorytmu uczenia.

Centralnym elementem jest klasa `UTHGame`. Pojedyncza instancja przechowuje stan aktualnego rozdania, źródło kart oraz opcjonalną historię decyzji. Po zakończeniu rozdania ta sama instancja może zostać wykorzystana ponownie przez wywołanie metody `reset()`.

3.9.1. Tworzenie silnika

Konstruktor klasy `UTHGame` przyjmuje dwa opcjonalne parametry:

- `seed` – ziarno nadrzędnego generatora liczb pseudolosowych,
- `record_history` – informację, czy należy rejestrować przebieg kolejnych decyzji.

Ziarno określa powtarzalny strumień talii generowanych dla kolejnych rozdawań. Parametr nie ustala pojedynczej, niezmienniej talii dla całej instancji. Każde wywołanie `reset()` tworzy nową talię, której ziarno jest pobierane z generatora środowiska.

Rejestrowanie historii jest domyślnie wyłączone. Dzięki temu podstawowe wykonywanie dużej liczby rozdań nie wymaga przechowywania dodatkowych obiektów diagno-

stycznych.

3.9.2. Rozpoczęcie epizodu

Metoda `reset()` rozpoczyna nowe rozdanie i zwraca pierwszą obserwację w fazie PREFLOP. W standardowym użyciu środowisko samodzielnie tworzy przetasowaną talię. Metoda umożliwia również przekazanie własnego obiektu implementującego interfejs `CardSource`, na przykład deterministycznej talii `FixedDeck`.

Rozpoczęcie rozdania obejmuje wybór lub utworzenie źródła kart, rozdanie po dwie karty graczowi i krupierowi, utworzenie stanu PREFLOP, wyzerowanie historii bieżącego rozdania oraz utworzenie pierwszej obserwacji. Reset jest wykonywany transakcyjnie. Silnik najpierw próbuje pobrać wszystkie karty potrzebne do utworzenia stanu początkowego, a dopiero po powodzeniu zapisuje nowy stan i źródło kart. Jeżeli przekazane źródło nie może dostarczyć wymaganych kart, poprzedni stan silnika nie zostaje zastąpiony stanem częściowo utworzonego rozdania.

3.9.3. Wykonanie decyzji

Metoda `step()` przyjmuje jedną wartość typu `Action`, wykonuje odpowiadające jej przejście i zwraca obiekt `StepResult`, zawierający:

- obserwację po wykonaniu akcji,
- wynik rozdania, jeżeli osiągnięto stan terminalny,
- rozliczenie zakładów w stanie terminalnym,
- szczegóły showdownu, jeżeli rozdanie nie zakończyło się foldem.

Wynik kroku pośredniego zawiera jedynie kolejną obserwację. Pola dotyczące wyniku i rozliczenia pozostają wtedy nieokreślone. W kroku terminalnym obiekt zawiera komplet informacji niezbędnych do obliczenia nagrody i zarejestrowania wyniku eksperymentu.

Właściwość `terminated` obiektu `StepResult` informuje, czy wykonana akcja zakończyła rozdanie. Jej wartość jest wyznaczana na podstawie fazy zawartej w obserwacji.

3.9.4. Właściwości publiczne

Poza metodami `reset()` i `step()` silnik udostępnia właściwości przedstawione w tabeli 3.7.

Właściwość `state` nie jest częścią interfejsu przeznaczonego do podejmowania decyzji przez agenta. Udostępniono ją na potrzeby testów, diagnostyki i analizy środowiska. Agent otrzymuje wartość właściwości `observation` albo obserwację zwróconą przez `reset()` lub `step()`.

Tabela 3.7. Publiczny interfejs klasy UTHGame

Element	Typ wyniku	Znaczenie
reset()	UTHObservation	Rozpoczyna nowe rozdanie i zwraca obserwację preflop.
step(action)	StepResult	Wykonuje legalną akcję i zwraca wynik przejścia.
observation	UTHObservation	Zwraca aktualne informacje widoczne dla agenta.
state	RoundState	Zwraca pełny stan wewnętrzny, przeznaczony głównie do testów i diagnostyki.
is_started	bool	Informuje, czy rozpoczęto co najmniej jedno rozdanie.
history_enabled	bool	Informuje, czy rejestrowanie historii jest aktywne.
trace	RoundTrace lub None	Zwraca zapis bieżącego rozdania, jeżeli historia jest włączona.

3.10. Rejestrowanie przebiegu i walidacja środowiska

Poprawność środowiska ma bezpośredni wpływ na wyniki eksperymentów. Błąd w kolejności rozdawania kart, rozliczaniu zakładów albo udostępnianiu obserwacji mógłby zostać błędnie zinterpretowany jako zachowanie wyuczone przez agenta. Z tego względu implementację uzupełniono o opcjonalny mechanizm rejestrowania przebiegu rozdania oraz zestaw testów weryfikujących zarówno pojedyncze komponenty, jak i pełne ścieżki rozgrywki.

3.10.1. Opcjonalne rejestrowanie historii

Rejestrowanie historii jest aktywowane parametrem `record_history` przekazywanym podczas tworzenia obiektu `UTHGame`. Domyślnie mechanizm pozostaje wyłączony, aby standardowe wykonywanie dużej liczby epizodów nie wymagało tworzenia dodatkowych obiektów diagnostycznych.

Po włączeniu historii każde poprawnie rozpoczęte rozdanie otrzymuje dodatni, rosnący identyfikator `round_id`. Lista decyzji jest zerowana przy każdym poprawnym wywołaniu metody `reset()`.

Właściwość `trace` zwraca obiekt `RoundTrace`. Jeżeli historia jest wyłączona albo nie rozpoczęto jeszcze rozdania, jej wartością jest `None`.

Mechanizm historii nie stanowi pamięci agenta i nie jest częścią obserwacji. Jest przeznaczony do m.in: diagnozowania nieoczekiwanych decyzji, odtwarzania przebiegu wybranych rozdań czy weryfikowania zgodności decyzji z legalną przestrzenią akcji.

3.10.2. Rekordy historii i ich transakcyjny zapis

Pojedynczą decyzję reprezentuje niemodyfikowalny obiekt `DecisionRecord`, zawierający obserwację dostępną agentowi przed wykonaniem decyzji oraz wybraną akcją. Zapisanie obserwacji sprzed przejścia pozwala odtworzyć informacje, na podstawie któ-

rych agent rzeczywiście podjął decyzję, a zapis obserwacji po akcji byłby niejednoznaczny, ponieważ mógłby zawierać karty odkryte dopiero jako jej skutek. Podczas tworzenia rekordu sprawdzane jest, że obserwacja nie jest terminalna, a akcja należy do zbioru legalnego dla tej obserwacji.

Cały przebieg rozdania opisuje niemodyfikowalny obiekt `RoundTrace`, zawierający identyfikator rozdania, uporządkowaną krotkę rekordów decyzji oraz aktualny pełny stan `RoundState`. Obserwacje w rekordach zawierają wyłącznie informacje dostępne agentowi, natomiast końcowy stan jest pełnym obiektem diagnostycznym, z tego powodu `RoundTrace` nie jest przekazywany agentowi, lecz służy testom i narzędziom analitycznym. Udostępnia też właściwości pomocnicze `completed`, `outcome`, `settlement` i `showdown`, które przed zakończeniem rozdania mogą zwracać `None`.

Zapis historii jest transakcyjny. Metoda `step()` zapamiętuje bieżącą obserwację, wykonuje walidację i przejście, a rekord dodaje dopiero po utworzeniu poprawnego kolejnego stanu. Jeżeli walidacja lub odkrywanie kart zakończy się wyjątkiem, rekord nie powstaje, więc historia nie zawiera akcji nielegalnych, decyzji po zakończeniu rozdania ani niedokończonych przejść. Tę samą zasadę stosuje metoda `reset()`, która zapisuje nowy stan dopiero po rozdaniu wszystkich czterech kart początkowych.

3.10.3. Poziomy testowania

Testy podzielono według odpowiedzialności komponentów. Testy jednostkowe sprawdzają w izolacji model kart i talię, evaluator, modele stanu, rozdawanie, legalność akcji, `showdown` oraz rozliczenia. Testy integracyjne wykonują kompletne rozdania obejmujące wszystkie ścieżki maszyny stanów, od `reset()` do stanu terminalnego, wraz z rejestrowaniem historii. Samo poprawne przetestowanie evaluatora i modułu rozliczeń nie gwarantuje bowiem, że silnik przekaże im właściwe karty i mnożnik zakładu `Play`.

Poza przykładowymi wynikami testy weryfikują niezmienniki obowiązujące dla wszystkich poprawnych rozdań, między innymi brak powtarzających się kart, zgodność liczby kart wspólnych i spalonych z fazą, brak danych terminalnych w stanie pośrednim, brak `showdownu` po spasowaniu, wykonanie co najwyżej jednego zakładu `Play`, brak kart krupiera i kart spalonych w obserwacji oraz brak zmiany stanu po odrzuconej akcji. Walidacja obejmuje zatem także przypadki brzegowe i próby utworzenia stanów naruszających reguły domenowe.

3.10.4. Pokrycie kodu

Zestaw testów uruchamiany jest za pomocą biblioteki `pytest`. Podczas prac nad silnikiem analizowano zarówno pokrycie instrukcji, jak i pokrycie rozgałęzień. Pozwoliło to wykryć nieprzetestowane ścieżki walidacji, przypadki brzegowe evaluatora oraz alternatywne sposoby rozliczania zakładów.

Na etapie zakończenia implementacji podstawowego silnika uzyskano pełne po-

krycie instrukcji i rozgałęzień dla objętych pomiarem modułów. Wynik pokrycia nie stanowi samodzielnego dowodu poprawności implementacji. Informuje jedynie, że testy wykonały wszystkie mierzone linie i gałęzie programu. Z tego względu został uzupełniony testami opartymi na konkretnych scenariuszach domenowych oraz walidacją niezmienników.

3.11. Ograniczenia środowiska

Zaimplementowane środowisko odwzorowuje podstawowy wariant Ultimate Texas Hold'em potrzebny do badania strategii podejmowania decyzji, a nie pełną symulację stołu kasynowego. Jego zakres celowo ograniczono do elementów wpływających na decyzje Play i końcowy wynik zakładów Ante, Blind i Play. Poza tym zakresem pozostają zakłady boczne Trips i Progressive, model wielu graczy przy stole, saldo gracza i limity stołu, interfejs graficzny.

Zakłady Trips i Progressive nie wprowadzają dodatkowego momentu decyzyjnego, ponieważ ich wynik nie zależy od sekwencji akcji Play. Wpływają jednak na rozkład końcowych wypłat i utrudniałyby interpretację jakości strategii, a Progressive wymagałby dodatkowo modelu puli utrzymywanej pomiędzy rozdaniem, co naruszałoby niezależność epizodów. Z tego powodu ocena dotyczy wyłącznie zakładów Ante, Blind i Play. Oba zakłady boczne można w razie potrzeby dołączyć jako rozszerzenie warstwy rozliczeń, bez zmiany kolejności faz.

Silnik nie modeluje salda gracza, sesji, ryzyka bankructwa ani zarządzania kapitałem, a wartości zakładów wyraża w jednostkach Ante, dzięki czemu strategię można porównywać niezależnie od nominału. Środowisko reprezentuje pojedynczego gracza wobec krupiera i nie symuluje pozostałych miejsc przy stole, a zakryte karty innych graczy i tak byłyby nieznane, więc rozkład kart widocznych dla gracza odpowiada losowaniu bez zwracania ze standardowej talii.

Obserwacja pozostaje obiektem domenowym zawierającym karty, fazę i zbiór legalnych akcji. Taka postać jest czytelna i wygodna w testach, lecz nie może być bezpośrednio podana modelom uczenia maszynowego. Kodowanie `UTHObservation` do reprezentacji numerycznej pozostaje poza zakresem klasy `UTHGame`, ponieważ porównanie reprezentacji stanu jest jednym z elementów badawczych pracy. Wspólny silnik pozwala zestawiać różne reprezentacje na identycznych rozdaniach i regułach.

3.12. Podsumowanie

W rozdziale przedstawiono projekt i implementację środowiska pojedynczego rozdania Ultimate Texas Hold'em. Silnik podzielono na niezależne komponenty odpowiedzialne za reprezentację kart, ocenę układów, rozdawanie, przechowywanie stanu, walidację akcji, showdown oraz rozliczanie zakładów, a przebieg rozdania odwzorowano jako maszynę stanów obejmującą fazy PREFLOP, FLOP, RIVER i TERMINAL, w której każda

akcja BET kończy część decyzyjną i wymusza co najwyżej jeden zakład Play.

Kluczowym elementem projektu jest rozdzielenie pełnego stanu RoundState od obserwacji UTHObservation: agent otrzymuje wyłącznie własne karty, odkryte karty wspólne, fazę i legalne akcje, podczas gdy karty krupiera, karty spalone i kolejność talii pozostają wewnętrzne. Showdown korzysta z ogólnego ewaluatora wybierającego najlepsze pięć kart z siedmiu, a rozliczenia Ante, Blind i Play są przechowywane dokładnie za pomocą typu Fraction. Deterministyczne źródło FixedDeck oraz opcjonalny mechanizm historii zapewniają powtarzalność i możliwość analizy przebiegu rozdań.

Środowisko stanowi warstwę domenową projektu. Gotowy silnik zapewnia podstawę do implementacji agentów bazowych, infrastruktury eksperymentalnej oraz metod uczenia przez wzmacnianie analizowanych w dalszej części pracy.

4. Agenci bazowi i infrastruktura eksperymentalna

W niniejszym rozdziale przedstawiono projekt agentów bazowych oraz infrastruktury służącej do uruchamiania i oceny strategii w środowisku Ultimate Texas Hold'em. Strategie bazowe pełnią rolę punktów odniesienia dla metod uczenia przez wzmacnianie. Pozwalają sprawdzić poprawność procesu prowadzenia eksperymentów oraz określić, czy wyuczona polityka przewyższa proste sposoby wyboru akcji.

Zaimplementowano wspólny kontrakt agenta, mechanizmy prowadzenia pojedynczego epizodu i symulacji wielu rozdań, agenta losowego oraz deterministycznego agenta regułowego. Infrastrukturę uzupełniono o mechanizm agregacji metryk i zapis wyników eksperymentów. Wszystkie te elementy korzystają z publicznego interfejsu środowiska opisanego w rozdziale 3.

4.1. Wspólny interfejs agenta

Wspólny kontrakt agentów zdefiniowano za pomocą protokołu `Agent`. Każda zgodna implementacja udostępnia metodę `select_action()`, która przyjmuje obiekt `UTHObservation` i zwraca jedną wartość typu `Action`.

Zastosowanie protokołu pozwala korzystać z typowania strukturalnego. Klasa agenta nie musi dziedziczyć po wspólnej klasie bazowej, lecz musi udostępniać metodę o wymaganej sygnaturze. Dzięki temu ten sam mechanizm prowadzenia rozgrywki może obsługiwać agentów losowych, regułowych i uczących się bez znajomości ich wewnętrznej implementacji.

Agent otrzymuje wyłącznie obserwację zawierającą informacje dostępne graczowi. Pełny obiekt `RoundState` nie jest częścią interfejsu, dlatego implementacja strategii nie uzyskuje dostępu do kart krupiera, kart spalonych ani kolejności pozostałych kart w talii.

4.2. Prowadzenie pojedynczego epizodu

Funkcja `play_round()` łączy agenta z silnikiem `UTHGame`. Rozpoczyna rozdanie, przekazuje bieżącą obserwację agentowi, wykonuje wybraną akcję i powtarza ten proces do osiągnięcia stanu terminalnego.

Każda decyzja zaakceptowana przez środowisko zostaje zapisana w kolejności wykonania. Po zakończeniu rozdania funkcja zwraca obiekt `EpisodeResult`, który zawiera sekwencję akcji, wynik rozdania oraz pełne rozliczenie zakładów Ante, Blind i Play.

Model `EpisodeResult` udostępnia również wynik netto, łączną wartość postawionych zakładów, liczbę decyzji i mnożnik zakładu Play. Stanowi w ten sposób lekką reprezentację zakończonego epizodu, odpowiednią do późniejszej agregacji wyników wielu rozdań.

4.3. Agent losowy

Agent losowy stanowi najprostszy punkt odniesienia. Nie analizuje wartości kart ani prawdopodobieństwa wygranej. Dla każdej obserwacji wybiera jedną z legalnych akcji.

Implementacja korzysta wyłącznie ze zbioru legalnych akcji zawartego w `UTHObservation`. Akcje są porządkowane zgodnie z kolejnością wartości enuma `Action`, dzięki czemu wynik generatora pseudolosowego nie zależy od kolejności iterowania po zbiorze.

4.3.1. Rozdzielenie źródeł losowości

Środowisko i agent posiadają niezależne generatory pseudolosowe. Generator środowiska odpowiada za kolejność kart, natomiast generator agenta odpowiada wyłącznie za wybór akcji:

$$seed_{environment} \rightarrow \text{kolejność kart}, \quad seed_{agent} \rightarrow \text{wybór akcji}. \quad (4.1)$$

Rozdzielenie obu źródeł losowości umożliwia zmianę strategii agenta bez zmiany przebiegu rozdań. Pozwala również odtwarzać sekwencje decyzji agenta niezależnie od konfiguracji środowiska.

4.3.2. Walidacja implementacji

Testy agenta losowego obejmują zgodność ze wspólnym protokołem, wybieranie wyłącznie legalnych akcji, powtarzalność decyzji dla tego samego seedu oraz obsługę niepoprawnych danych wejściowych. Osobny test potwierdza, że agent nie podejmuje decyzji dla obserwacji terminalnej.

Testy runnera obejmują wszystkie legalne ścieżki zakończenia rozdania. Sprawdzają również, czy agent otrzymuje obserwacje odpowiadające kolejnym fazom oraz czy jawnie przekazane źródło kart pozwala odtworzyć wynik niezależnie od wewnętrznego seedu obiektu `UTHGame`.

4.4. Agent regułowy

Drugą strategię bazową stanowi agent regułowy. W przeciwieństwie do agenta losowego podejmuje on decyzje na podstawie kart widocznych w obserwacji oraz aktualnej fazy rozdania. Przyjęte reguły oparto na uproszczonej strategii Ultimate Texas Hold'em publikowanej przez Wizard of Odds [3].

Strategia została podzielona na trzy zbiory reguł odpowiadające kolejnym punktom decyzyjnym środowiska.

4.4.1. Decyzja przed flopem

Przed odkryciem kart wspólnych agent wykonuje zakład Play równy czterokrotności Ante dla następujących układów początkowych:

- każdej pary od trójek wzwyż,

- każdego układu zawierającego asa,
- króla z dowolną kartą w tym samym kolorze,
- króla z kartą pięć lub wyższą w różnych kolorach,
- damy z kartą sześć lub wyższą w tym samym kolorze,
- damy z kartą osiem lub wyższą w różnych kolorach,
- waleta z kartą osiem lub wyższą w tym samym kolorze,
- waleta z dziesiątką w różnych kolorach.

Para dwójek oraz wszystkie pozostałe układy prowadzą do wykonania akcji CHECK. Przyjęta strategia nie wykorzystuje zakładu BET_3X, mimo że akcja ta pozostaje legalną akcją środowiska.

4.4.2. Decyzja po flopie

Jeżeli gracz nie wykonał zakładu przed flopem, po odkryciu trzech kart wspólnych agent wybiera BET_2X, gdy posiada dwie pary lub silniejszy układ, ukrytą parę z wyjątkiem pary dwójek albo cztery karty do koloru z własną kartą tego koloru o randze co najmniej dziesięć.

Przez ukrytą parę rozumiana jest para wykorzystująca co najmniej jedną z dwóch prywatnych kart gracza. Jeżeli żaden z wymienionych warunków nie jest spełniony, agent wykonuje akcję CHECK.

4.4.3. Decyzja po odkryciu wszystkich kart wspólnych

W ostatnim punkcie decyzyjnym agent wykonuje zakład BET_1X, jeżeli posiada ukrytą parę lub silniejszy układ. Dla słabszych układów wykorzystywana jest reguła oparta na liczbie kart mogących poprawić układ krupiera do układu pokonującego gracza. Jeżeli liczba takich kart jest mniejsza niż 21, agent wykonuje BET_1X. W przeciwnym przypadku wybiera FOLD [3].

Tak zdefiniowana strategia jest deterministyczna. Dla tej samej obserwacji agent zawsze zwraca tę samą akcję. Dzięki temu może stanowić silniejszy punkt odniesienia niż agent losowy, jednocześnie nie wykorzystując procesu uczenia ani informacji ukrytych przed graczem.

4.4.4. Implementacja strategii regułowej

Reguły decyzyjne zostały rozdzielone od klasy realizującej wspólny interfejs agenta. Funkcje `should_raise_preflop()`, `should_raise_flop()` oraz `should_raise_river()` odpowiadają za ocenę obserwacji w poszczególnych fazach rozdania. Klasa `RuleBasedAgent` wybiera na ich podstawie akcję zgodną z aktualną fazą gry.

Dla decyzji na riverze zaimplementowano dodatkową funkcję `count_dealer_outs()`. Tworzony jest zbiór kart niewidocznych dla gracza, a następnie każda z nich jest analizowana jako pojedyncza potencjalna karta prywatna krupiera. Jeżeli dołączenie danej karty do kart wspólnych prowadzi do układu silniejszego od najlepszego układu gracza, karta zostaje zaklasyfikowana jako out.

Podjęcie to odzwierciedla uproszczoną regułę 21 outów. Nie jest ono pełnym przeszukaniem wszystkich możliwych dwukartowych układów krupiera. Uwzględniane są pojedyncze niewidoczne karty, które same prowadzą do układu pokonującego gracza. Pozwala to zachować charakter strategii bazowej opisanej przez Wizard of Odds [3], zamiast zastępować ją dokładniejszą metodą obliczeniową.

Klasa `RuleBasedAgent` nie przechowuje własnego stanu i nie korzysta z generatora liczb pseudolosowych. Otrzymuje wyłącznie obiekt `UTHObservation`, dzięki czemu nie posiada dostępu do kart krupiera ani pozostałych informacji ukrytych w pełnym stanie środowiska.

4.5. Symulacja wielu epizodów

Do prowadzenia eksperymentów obejmujących wiele rozdań wykorzystano obiekty `SimulationConfig` i `SimulationResult`. Konfiguracja symulacji przechowuje uporządkowaną sekwencję seedów wykorzystywanych do tworzenia talii. Liczba seedów określa jednocześnie liczbę rozgrywanych epizodów.

Funkcja `run_simulation()` dla każdego seedu tworzy osobną talię oraz nową instancję środowiska `UTHGame`. Następnie wykorzystuje funkcję `play_round()` do rozegrania pełnego epizodu. Wyniki wszystkich rozdań są przechowywane w obiekcie `SimulationResult`.

Takie rozwiązanie oddziela konfigurację eksperymentu od implementacji konkretnego agenta. Ten sam obiekt `SimulationConfig` może zostać wykorzystany do oceny wielu różnych strategii.

4.6. Powtarzalność eksperymentów

Kluczowym założeniem infrastruktury eksperymentalnej jest możliwość odtworzenia przeprowadzonych symulacji. Dla każdego epizodu zapisywany jest osobny seed talii. Ponowne utworzenie talii z tym samym seedem prowadzi do tej samej kolejności kart.

Podczas porównywania agentów wykorzystywana jest identyczna sekwencja seedów talii. Oznacza to, że każdy agent jest oceniany na tym samym harmonogramie rozdań. Dzięki temu różnice wyników nie są skutkiem wylosowania niezależnych zestawów kart dla poszczególnych strategii.

W przypadku agenta losowego dodatkowo wykorzystywany jest niezależny seed odpowiedzialny wyłącznie za wybór akcji. Seed środowiska i seed agenta nie wpływają więc na siebie. Agent regułowy jest deterministyczny i nie wymaga własnego źródła losowości.

Konfiguracja wykorzystana w eksperymencie jest zapisywana razem z wynikami. Obejmuje ona liczbę rozdań, seed generatora harmonogramu talii, seed agenta losowego oraz pełną sekwencję seedów poszczególnych talii. Pozwala to ponownie przeprowadzić dokładnie skonfigurowany eksperyment.

4.7. Metryki oceny

Podstawową miarą jakości strategii jest wynik netto wyrażony w jednostkach zakładu Ante. Dla symulacji składającej się z N epizodów empiryczną wartość oczekiwaną oszacowano jako średni wynik netto na jedno rozdanie:

$$\widehat{EV} = \frac{1}{N} \sum_{i=1}^N r_i, \quad (4.2)$$

gdzie r_i oznacza wynik netto i -tego epizodu wyrażony w jednostkach Ante.

Oprócz średniego wyniku obliczane są całkowity wynik netto oraz średnia i całkowita wartość postawionych zakładów. Zmienność wyników opisano za pomocą odchylenia standardowego próby:

$$s = \sqrt{\frac{\sum_{i=1}^N (r_i - \bar{r})^2}{N - 1}}. \quad (4.3)$$

Na jego podstawie wyznaczany jest błąd standardowy średniej:

$$SE = \frac{s}{\sqrt{N}}. \quad (4.4)$$

Infrastruktura zlicza również końcowe wyniki rozdań oraz częstotliwość wykonywania poszczególnych akcji. Wyniki finansowe pojedynczych epizodów i ich agregaty przechowywane są z wykorzystaniem typu `Fraction`. Konwersja do liczb zmiennoprzecinkowych wykonywana jest dopiero dla statystyk wymagających takiej reprezentacji oraz podczas eksportu wyników.

4.8. Porównanie agentów bazowych

W celu zweryfikowania kompletnego procesu eksperymentalnego przeprowadzono porównanie agenta losowego i agenta regułowego. Każdy agent rozegrał 10 000 rozdań wykorzystujących ten sam harmonogram talii. Seed generatora harmonogramu wyniósł 20260815, natomiast generator decyzji agenta losowego zainicjalizowano seedem 20260816.

Uzyskane podstawowe metryki przedstawiono w tabeli 4.1.

Tabela 4.1. Wyniki agentów bazowych dla 10 000 rozdań

Agent	$\widehat{EV}$	s	SE	Śr. stawka	Wynik netto
RandomAgent	-0,46435	4,46422	0,04464	4,7395	-4643,5
RuleBasedAgent	0,03865	4,08130	0,04081	4,1905	386,5

Agent losowy uzyskał średni wynik $-0,46435$ jednostki Ante na rozdanie. Agent regułowy osiągnął w tej samej próbie średni wynik $0,03865$, czyli wynik wyższy o około $0,503$ jednostki Ante na rozdanie. Jednocześnie średnia wartość środków stawianych przez agenta regułowego była niższa.

Dodatni wynik agenta regułowego w tej konkretnej próbie nie powinien być interpretowany jako dowód dodatniej rzeczywistej wartości oczekiwanej strategii. Błąd standardowy jego średniego wyniku wyniósł około $0,04081$, czyli wartość zbliżoną do samego oszacowania $\widehat{EV}$. Wynik eksperymentu służy przede wszystkim jako punkt odniesienia dla późniejszej oceny agentów uczących się.

Rozkład wyników końcowych przedstawiono w tabeli 4.2.

Tabela 4.2. Rozkład wyników rozdań agentów bazowych

Agent	Wygrana	Wygrana krupiera	Fold	Remis
RandomAgent	45,01%	42,75%	8,49%	3,75%
RuleBasedAgent	47,92%	31,75%	16,79%	3,54%

Agent regułowy częściej kończył rozdania foldem, lecz znacznie rzadziej doprowadzał do rozstrzygnięcia zakończonego wygraną krupiera. Wyniki te pokazują, że zastosowane reguły prowadzą do jakościowo innego sposobu podejmowania decyzji niż równomierny losowy wybór spośród legalnych akcji.

4.9. Zapis wyników eksperymentu

Eksperyment porównujący agentów bazowych zapisuje wyniki w dwóch formatach. Plik JSON zawiera konfigurację eksperymentu oraz zagregowane metryki obu agentów. Plik CSV przechowuje dane poszczególnych epizodów, w tym indeks rozdania, seed talii, nazwę agenta, wynik netto, całkowitą stawkę, wynik końcowy i sekwencję wykonanych akcji.

Zapis wyników na poziomie pojedynczego epizodu umożliwia późniejsze analizowanie przebiegu eksperymentu bez konieczności ponownego uruchamiania symulacji. Pozwala również zestawiać wyniki obu agentów dla tych samych seedów talii oraz tworzyć dodatkowe statystyki i wizualizacje.

4.10. Walidacja infrastruktury eksperymentalnej

Testy infrastruktury obejmują zarówno pojedyncze epizody, jak i symulacje wielu rozdań. Sprawdzana jest zgodność liczby epizodów z konfiguracją, powtarzalność wyników dla tych samych seedów oraz wykorzystanie tego samego harmonogramu talii przez porównywanych agentów.

Testy agregacji metryk weryfikują obliczanie wyniku całkowitego, średniego wyniku, wartości postawionych zakładów, odchylenia standardowego, błędu standardowego oraz liczników wyników i akcji. Osobne testy obejmują zapis wyników eksperymentu i sprawdzają, czy zapisany harmonogram talii odpowiada konfiguracji symulacji oraz czy

dla obu agentów zapisano właściwą liczbę epizodów.

Takie rozdzielanie odpowiedzialności pozwala niezależnie testować silnik gry, strategie agentów, prowadzenie epizodów, agregację metryk oraz warstwę odpowiedzialną za zapis eksperymentów.

4.11. Podsumowanie

W rozdziale zdefiniowano dwa punkty odniesienia dla dalszych eksperymentów. `RandomAgent` reprezentuje strategię pozbawioną wiedzy o wartości kart, natomiast `RuleBasedAgent` wykorzystuje deterministyczny zestaw reguł oparty na uproszczonej strategii Ultimate Texas Hold'em.

Przygotowana infrastruktura umożliwia prowadzenie powtarzalnych symulacji na wspólnych harmonogramach rozdań, agregowanie wyników i zapisywanie danych poszczególnych epizodów. Stanowi ona podstawę do porównywania agentów wykorzystujących metody uczenia przez wzmacnianie z ustalonymi strategiami bazowymi w kolejnych etapach pracy.

5. Metody uczenia przez wzmacnianie

W niniejszym rozdziale przedstawiono elementy warstwy uczenia przez wzmacnianie budowanej nad środowiskiem Ultimate Texas Hold'em. Obejmują one sposób przekształcania obserwacji do stałowymiarowej reprezentacji numerycznej oraz definicję sygnału nagrody wykorzystywanego później przez algorytmy uczące się.

Środowisko udostępnia agentowi obiekt `UTHObservation`. Zawiera on wyłącznie informacje dostępne graczowi, w szczególności jego dwie karty prywatne, odkryte karty wspólne, aktualną fazę rozdania oraz zbiór legalnych akcji. Reprezentacja numeryczna jest budowana wyłącznie na podstawie tego obiektu, dzięki czemu proces uczenia nie uzyskuje dostępu do kart krupiera ani innych informacji ukrytych w pełnym stanie środowiska.

5.1. Reprezentacja stanu

W eksperymentach przewidziano dwa warianty reprezentacji obserwacji. Pierwszy wariant stanowi reprezentacja surowa, która koduje informacje dostępne w `UTHObservation` bez dodawania ręcznie zaprojektowanych cech pokerowych. Drugi wariant rozszerza reprezentację surową o cechy domenowe wyznaczane na podstawie kart widocznych dla gracza.

Rozdzielenie reprezentacji od implementacji konkretnych algorytmów pozwala wykorzystać ten sam sposób kodowania zarówno w agencie wykorzystującym Deep Q-Network, jak i w agencie opartym na metodzie Policy Gradient. Dzięki temu wpływ reprezentacji stanu może być analizowany niezależnie od wybranego algorytmu uczenia.

5.1.1. Wspólny interfejs kodowania

Warstwa reprezentacji definiuje protokół `StateEncoder`. Każdy encoder udostępnia metodę `encode()`, która przyjmuje obiekt `UTHObservation` i zwraca wektor wartości zmiennoprzecinkowych. Dodatkowa właściwość `output_size` określa stały wymiar generowanego wektora.

Wspólny kontrakt ma postać

$$f_{\text{enc}}: O_t \rightarrow \mathbb{R}^d, \quad (5.1)$$

gdzie O_t oznacza obserwację w chwili t , natomiast d jest wymiarem wybranej reprezentacji.

Encoder nie zależy od biblioteki wykorzystywanej później do budowy sieci neuronowych. Wynikiem kodowania jest zwykły wektor numeryczny, który może zostać następnie przekształcony do formatu wymaganego przez konkretny algorytm uczenia.

5.2. Surowa reprezentacja obserwacji

Pierwszy wariant reprezentacji został zaimplementowany w klasie `RawStateEncoder`. Jego zadaniem jest zachowanie informacji zawartych bezpośrednio w obserwacji bez wprowadzania jawnych informacji o sile układu pokerowego, drawach lub strukturze stołu.

Reprezentacja składa się z czterech części:

1. zakodowanych dwóch kart prywatnych gracza,
2. pięciu slotów przeznaczonych na karty wspólne,
3. kodowania aktualnej fazy rozdania,
4. maski legalnych akcji.

5.2.1. Kodowanie kart

Standardowa talia zawiera 52 unikalne karty. Każda karta jest reprezentowana przez wektor one-hot o długości 52. Dokładnie jedna pozycja wektora przyjmuje wartość 1, natomiast pozostałe pozycje mają wartość 0.

Karty uporządkowano najpierw według koloru, a następnie według rangi. Taki porządek jest deterministyczny i pozostaje zgodny ze sposobem tworzenia standardowej talii wykorzystywanej przez środowisko.

Dla karty c jej reprezentację można zapisać jako

$$x_c = [x_0, x_1, \dots, x_{51}], \quad (5.2)$$

gdzie

$$x_i = \begin{cases} 1, & i = \text{index}(c), \\ 0, & i \neq \text{index}(c). \end{cases} \quad (5.3)$$

W obserwacji zawsze występują dwie karty prywatne gracza. Dla kart wspólnych zarezerwowano natomiast pięć pozycji niezależnie od aktualnej fazy rozdania. Nieodkryte jeszcze karty wspólne są reprezentowane przez zerowy wektor długości 52.

Takie rozwiązanie zapewnia stały wymiar reprezentacji przed flopem, po flopie oraz po odkryciu wszystkich kart wspólnych.

5.2.2. Kodowanie fazy rozdania

Uwzględniane są trzy fazy, w których agent może podjąć decyzję: PREFLOP, FLOP oraz RIVER. Faza jest reprezentowana przez trzelementowy wektor one-hot.

Odpowiednie kodowania mają postać

$$\begin{aligned}
\text{PREFLOP} &\rightarrow [1, 0, 0], \\
\text{FLOP} &\rightarrow [0, 1, 0], \\
\text{RIVER} &\rightarrow [0, 0, 1].
\end{aligned}
\tag{5.4}$$

Stan terminalny nie jest kodowany przez `RawStateEncoder`, ponieważ nie wymaga już podjęcia kolejnej decyzji przez agenta.

5.2.3. Maska legalnych akcji

Ostatnim elementem reprezentacji jest maska legalnych akcji. Środowisko definiuje sześć możliwych wartości typu `Action`. Dla każdej z nich zapisywana jest wartość 1, jeżeli akcja jest legalna w aktualnej fazie, oraz 0 w przeciwnym przypadku.

Maska ma więc postać

$$m_t \in \{0, 1\}^6. \tag{5.5}$$

Jej obecność pozwala przyszłym agentom jednoznacznie rozpoznać zbiór akcji dostępnych w danym punkcie decyzyjnym. Mechanizm ten będzie również wykorzystywany podczas wyboru akcji przez modele uczące się.

5.2.4. Wymiar reprezentacji

Na kodowanie kart przeznaczono siedem slotów, z czego dwa odpowiadają kartom gracza, a pięć kartom wspólnym. Każdy slot ma długość 52. Dodatkowo reprezentacja zawiera trzy wartości opisujące fazę oraz sześć wartości maski akcji.

Całkowity wymiar reprezentacji wynosi zatem

$$d_{\text{raw}} = 7 \cdot 52 + 3 + 6 = 373. \tag{5.6}$$

Wymiar pozostaje stały dla wszystkich stanów decyzyjnych. Jest to istotne z punktu widzenia późniejszych modeli neuronowych, ponieważ warstwa wejściowa sieci może posiadać z góry określoną liczbę elementów.

5.2.5. Charakter reprezentacji surowej

Reprezentacja surowa celowo nie zawiera informacji takich jak kategoria aktualnego układu, liczba kart do koloru, możliwość utworzenia strita ani innych cech wynikających z wiedzy pokerowej. Model otrzymuje jedynie zakodowane dane dostępne bezpośrednio w obserwacji.

Wariant ten stanowi punkt odniesienia dla reprezentacji wzbogaconej o cechy domenowe. Porównanie obu wariantów pozwoli ocenić, czy jawne dostarczenie modelowi informacji pokerowych wpływa na skuteczność i przebieg procesu uczenia.

5.3. Reprezentacja wzbogacona o cechy domenowe

Drugim wariantem reprezentacji jest reprezentacja wzbogacona o cechy domenowe. Została ona zaimplementowana w klasie `FeatureStateEncoder`. Jej podstawę stanowi pełny wektor generowany przez `RawStateEncoder`, do którego dołączany jest zestaw cech opisujących własności pokerowe kart widocznych dla gracza.

Reprezentację można zapisać jako

$$x_{\text{features}} = [x_{\text{raw}}, x_{\text{poker}}], \quad (5.7)$$

gdzie x_{raw} oznacza surową reprezentację o wymiarze 373, natomiast x_{poker} jest wektorem 20 cech domenowych.

Cechy domenowe są wyznaczane wyłącznie na podstawie kart prywatnych gracza oraz odkrytych kart wspólnych. Proces ekstrakcji nie korzysta z kart krupiera, kart spalonych ani kolejności kart pozostających w talii.

5.3.1. Cechy kart prywatnych

Pierwszych pięć wartości opisuje dwie prywatne karty gracza. Uwzględniono rangę wyższej karty, rangę niższej karty, informację o parze, zgodność kolorów oraz różnicę rang.

Rangi są normalizowane do przedziału od 0 do 1 zgodnie z zależnością

$$r_{\text{norm}} = \frac{r - 2}{12}, \quad (5.8)$$

gdzie r jest wartością rangi karty od 2 do 14.

Różnica rang kart prywatnych jest kodowana jako

$$g = \frac{|r_1 - r_2|}{12}. \quad (5.9)$$

Informacje o parze i zgodności kolorów są wartościami binarnymi.

5.3.2. Kategoria układu pokerowego

Kolejnych dziewięć wartości stanowi kodowanie one-hot kategorii najlepszego aktualnie dostępnego układu pokerowego. Uwzględniono kategorie od wysokiej karty do pokera:

- wysoka karta,
- jedna para,
- dwie pary,
- trójka,
- strit,

- kolor,
- full house,
- kareta,
- poker.

Po flopie dostępnych jest dokładnie pięć widocznych kart, dlatego możliwa jest bezpośrednia ocena pięciokartowego układu. Po odkryciu turna i rivera wybierany jest najlepszy układ pięciokartowy spośród siedmiu widocznych kart.

Przed flopem dostępne są jedynie dwie karty prywatne, dlatego kategoria pełnego układu pięciokartowego nie jest jeszcze zdefiniowana. Dziewięcioelementowy fragment reprezentacji przyjmuje wtedy same wartości zerowe.

5.3.3. Cechy strukturalne

Ostatnich sześć wartości opisuje strukturę kart widocznych dla gracza. Uwzględniono liczbę par, liczbę trójek, liczbę karek, postęp w kierunku koloru, postęp w kierunku strita oraz informację o sparowaniu kart wspólnych.

Liczba par jest normalizowana względem maksymalnej liczby trzech par możliwych wśród siedmiu kart:

$$p_{\text{pair}} = \frac{n_{\text{pair}}}{3}. \quad (5.10)$$

Analogicznie liczba trójek jest dzielona przez dwa, natomiast informacja o karecie przyjmuje bezpośrednio wartość 0 albo 1.

Postęp w kierunku koloru określono jako największą liczbę widocznych kart jednego koloru ograniczoną do pięciu i podzieloną przez pięć:

$$p_{\text{flush}} = \frac{\min(n_{\text{suit}}, 5)}{5}. \quad (5.11)$$

Wartość 0,8 oznacza przykładowo cztery widoczne karty tego samego koloru, natomiast 1 oznacza co najmniej pięć kart jednego koloru.

Postęp w kierunku strita obliczany jest przez porównanie widocznych rang ze wszystkimi możliwymi pięciokartowymi zbiorami rang tworzącymi strita. Wybierany jest zbiór mający największą liczbę wspólnych rang z kartami widocznymi, a uzyskana liczba dzielona jest przez pięć.

Dzięki temu dla widocznych rang

$$A, K, Q, J \quad (5.12)$$

wartość cechy wynosi

$$p_{\text{straight}} = \frac{4}{5} = 0,8. \quad (5.13)$$

Osobna cecha binarna określa, czy wśród odkrytych kart wspólnych występują co najmniej dwie karty tej samej rangi.

5.3.4. Wymiar reprezentacji wzbogaconej

Wektor cech domenowych składa się z pięciu cech kart prywatnych, dziewięciu wartości kodujących kategorię układu oraz sześciu cech strukturalnych:

$$d_{\text{poker}} = 5 + 9 + 6 = 20. \quad (5.14)$$

Po dołączeniu ich do reprezentacji surowej całkowity wymiar drugiego wariantu wynosi

$$d_{\text{features}} = 373 + 20 = 393. \quad (5.15)$$

5.4. Porównanie wariantów reprezentacji

Podstawowe różnice między oboma wariantami przedstawiono w tabeli 5.1.

Tabela 5.1. Porównanie reprezentacji stanu

Właściwość	State A	State B
Typ	surowa	wzbogacona
Wymiar	373	393
Kodowanie kart	one-hot	one-hot
Faza rozdania	tak	tak
Maska legalnych akcji	tak	tak
Cechy pokerowe	nie	20
Informacje ukryte	nie	nie

State A wymaga od modelu samodzielnego nauczenia się zależności między zakodowanymi kartami a strukturą układów pokerowych. State B dostarcza część tych zależności w postaci jawnych cech, wprowadzając do modelu wiedzę domenową.

Takie rozwiązanie pozwala zbadać wpływ indukcyjnego obciążenia wynikającego z wiedzy pokerowej. Reprezentacja wzbogacona może ułatwiać proces uczenia i zmniejszać liczbę interakcji potrzebnych do rozpoznawania istotnych struktur kart. Jednocześnie reprezentacja surowa w mniejszym stopniu narzuca modelowi sposób interpretacji obserwacji.

5.5. Konfiguracja reprezentacji

Wariant reprezentacji jest wybierany za pomocą typu `StateRepresentation`. Dostępne wartości to `RAW` oraz `FEATURES`.

Funkcja `build_state_encoder()` tworzy encoder odpowiadający wybranej konfiguracji. Dzięki temu przyszła implementacja algorytmu uczenia nie musi bezpośrednio

zależać od klas `RawStateEncoder` i `FeatureStateEncoder`.

Rozdzielenie wyboru reprezentacji od algorytmu umożliwi uruchamianie tych samych metod uczenia z różnymi sposobami kodowania obserwacji. Wybrany wariant może być również zapisany jako część konfiguracji eksperymentu za pomocą stabilnych wartości tekstowych `raw` i `features`.

5.6. Walidacja reprezentacji

Testy jednostkowe sprawdzają deterministyczność kodowania kart, stałość wymiarów obu reprezentacji, kodowanie poszczególnych faz rozdania, maskę legalnych akcji oraz poprawność cech domenowych.

Dodatkowo przeprowadzono test integracyjny z wykorzystaniem rzeczywistego przebiegu środowiska `UTHGame`. Obserwacje kolejnych faz `PREFLOP`, `FLOP` i `RIVER` są kodowane przez oba warianty reprezentacji.

Osobny test wykorzystuje dwa rozdania różniące się kartami krupiera i kartami spalonymi, lecz posiadające identyczne informacje widoczne dla gracza. W obu przypadkach uzyskane reprezentacje są identyczne. Potwierdza to, że warstwa kodowania nie wykorzystuje informacji ukrytych w pełnym stanie środowiska.

5.7. Hipoteza dotycząca reprezentacji stanu

W dalszych eksperymentach oba warianty zostaną wykorzystane z tymi samymi algorytmami uczenia i funkcjami nagrody. Pozwoli to ocenić wpływ reprezentacji stanu przy zachowaniu pozostałych elementów konfiguracji eksperymentu.

Przyjęto hipotezę, że wzbogacenie reprezentacji o jawne cechy pokerowe może przyspieszyć proces uczenia oraz zwiększyć stabilność uzyskiwanych strategii, ponieważ model nie musi samodzielnie rekonstruować części zależności domenowych z kodowania pojedynczych kart.

Jednocześnie nie zakłada się z góry, że reprezentacja wzbogacona prowadzi do wyższej końcowej wartości oczekiwanej strategii. Wpływ obu wariantów zostanie określony na podstawie wyników eksperymentalnych.

5.8. Funkcja nagrody

Drugim elementem wspólnym dla badanych metod uczenia przez wzmacnianie jest funkcja nagrody. Jej zadaniem jest przekształcenie wyniku pojedynczego kroku środowiska w wartość numeryczną wykorzystywaną podczas uczenia agenta.

Warstwa funkcji nagrody została oddzielona zarówno od reprezentacji stanu, jak i od implementacji konkretnego algorytmu uczenia. Zdefiniowany protokół `RewardFunction` przyjmuje obiekt `StepResult` zwracany przez środowisko i zwraca wartość zmiennoprzecinkową.

Kontrakt funkcji nagrody można zapisać jako

$$f_{\text{reward}}: S_t \rightarrow \mathbb{R}, \quad (5.16)$$

gdzie S_t oznacza wynik wykonania pojedynczej akcji reprezentowany przez obiekt `StepResult`.

Funkcja nagrody nie otrzymuje bezpośredniego dostępu do pełnego `RoundState`. Korzysta wyłącznie z wyniku kroku udostępnionego przez publiczny interfejs środowiska.

5.8.1. Terminalny charakter nagrody

W obu analizowanych wariantach zastosowano nagrodę terminalną. Dla kroków, które nie kończą rozdania, wartość nagrody jest równa zero:

$$r_t = 0 \quad \text{dla kroku nieterminalnego.} \quad (5.17)$$

Agent nie otrzymuje zatem dodatkowych nagród za wykonanie określonej akcji, uzyskanie silnego układu, kontynuowanie rozdania ani zrezygnowanie z dalszej gry.

Dopiero akcja kończąca rozdanie powoduje wygenerowanie nagrody wynikającej z finansowego rozliczenia zakładów.

Takie rozwiązanie zachowuje bezpośredni związek pomiędzy celem uczenia a rzeczywistym wynikiem ekonomicznym strategii w Ultimate Texas Hold'em.

5.9. Nagroda oparta na zysku netto

Pierwszym wariantem funkcji nagrody jest `NetProfitReward`. Dla zakończonego rozdania zwracany jest całkowity zysk lub strata netto gracza dostępna jako `Settlement.total_net_profit`.

Nagrodę można zapisać jako

$$R_{\text{net}} = \begin{cases} 0, & \text{gdy rozdanie trwa,} \\ P_{\text{net}}, & \text{gdy rozdanie zostało zakończone,} \end{cases} \quad (5.18)$$

gdzie P_{net} oznacza końcowy zysk netto wyrażony w jednostkach stawki Ante.

Przykładowo wartość -2 oznacza stratę dwóch jednostek Ante, natomiast wartość $5,5$ oznacza zysk pięciu i pół jednostki Ante.

Funkcja nagrody nie implementuje ponownie zasad wypłat zakładów Ante, Blind ani Play. Za ich rozliczenie odpowiada warstwa środowiska, a jedynym źródłem wartości ekonomicznej dla funkcji nagrody jest obiekt `Settlement`.

Pozwala to uniknąć sytuacji, w której środowisko oraz warstwa uczenia stosowałyby niezależne implementacje zasad wypłat.

5.9.1. Dokładność obliczeń finansowych

Środowisko reprezentuje wartości finansowe za pomocą typu `Fraction`. Dotyczy to między innymi wypłat zakładu Blind, które mogą przyjmować wartości ułamkowe.

Dokładna arytmetyka jest zachowywana przez cały proces rozliczenia rozdania. Konwersja do liczby zmiennoprzecinkowej następuje dopiero na granicy warstwy uczenia przez wzmacnianie, podczas wyznaczania wartości funkcji nagrody.

Takie rozdzielanie umożliwia zachowanie dokładności logiki finansowej środowiska przy jednoczesnym dostarczeniu wartości w formacie odpowiednim dla przyszłych modeli neuronowych.

5.10. Nagroda skalowana względem maksymalnej stawki

Drugim wariantem jest StakeScaledNetProfitReward. Stanowi on dodatnie liniowe przeskalowanie nagrody opartej na zysku netto.

W standardowym rozdaniu gracz zawsze stawia jedną jednostkę Ante oraz jedną jednostkę Blind. Największy możliwy zakład Play wynosi cztery jednostki Ante. Maksymalna standardowa wartość wszystkich postawionych środków wynosi więc

$$S_{\max} = 1 + 1 + 4 = 6. \quad (5.19)$$

Drugi wariant nagrody zdefiniowano jako

$$R_{\text{scaled}} = \frac{R_{\text{net}}}{6}. \quad (5.20)$$

Dla maksymalnej standardowej straty wynoszącej sześć jednostek Ante otrzymuje się zatem

$$R_{\text{scaled}} = \frac{-6}{6} = -1. \quad (5.21)$$

Dla spasowania przed postawieniem zakładu Play strata wynosi dwie jednostki Ante, dlatego

$$R_{\text{scaled}} = \frac{-2}{6} = -\frac{1}{3}. \quad (5.22)$$

Skalowanie wykorzystuje stałą wartość 6, a nie rzeczywistą kwotę postawioną w konkretnym rozdaniu. Dzięki temu funkcja pozostaje dodatnim liniowym przekształceniem pierwszego wariantu nagrody.

5.10.1. Znaczenie skalowania

Zastosowane skalowanie nie jest clippingiem ani normalizacją nagrody do przedziału od -1 do 1 .

Zakład Blind posiada własną tabelę wypłat. Silne układy pokerowe mogą prowadzić do zysków znacznie przekraczających wysokość początkowych stawek. W konsekwencji dodatnia wartość R_{scaled} może być większa od 1 .

Celem drugiego wariantu nie jest ograniczenie zakresu nagród, lecz zmniejszenie

typowej skali sygnału przekazywanego do algorytmu uczenia.

Ponieważ

$$R_{\text{scaled}} = \frac{1}{6} R_{\text{net}}, \quad (5.23)$$

transformacja zachowuje znak nagrody oraz uporządkowanie wyników. Jeżeli dla dwóch wyników a i b

$$R_{\text{net}}(a) < R_{\text{net}}(b), \quad (5.24)$$

to również

$$R_{\text{scaled}}(a) < R_{\text{scaled}}(b). \quad (5.25)$$

Oba warianty reprezentują więc ten sam ekonomiczny cel strategii. Różnią się skalą sygnału przekazywanego do procesu uczenia.

5.11. Porównanie wariantów funkcji nagrody

Podstawowe właściwości obu wariantów przedstawiono w tabeli 5.2.

Tabela 5.2. Porównanie wariantów funkcji nagrody

Właściwość	Reward A	Reward B
Typ	zysk netto	zysk skalowany
Kroki nieterminalne	0	0
Wynik terminalny	P_{net}	$P_{\text{net}}/6$
Jednostka bazowa	Ante	Ante / 6
Zachowanie znaku	tak	tak
Ten sam cel ekonomiczny	tak	tak
Ograniczenie do $[-1, 1]$	nie	nie

Reward A zachowuje naturalną skalę ekonomicznego wyniku rozdania. Reward B zmniejsza wartości przekazywane do algorytmu uczenia, pozostawiając relacje pomiędzy wszystkimi wynikami bez zmian.

Porównanie tych wariantów pozwoli zbadać, czy sama skala terminalnego sygnału nagrody wpływa na przebieg uczenia, jego stabilność oraz szybkość zbieżności.

5.12. Konfiguracja funkcji nagrody

Wariant funkcji nagrody jest wybierany za pomocą typu `RewardType`. Dostępne są wartości `NET_PROFIT` oraz `STAKE_SCALED_NET_PROFIT`.

Funkcja `build_reward_function()` tworzy implementację odpowiadającą wybranemu wariantowi.

Dzięki wspólnemu protokołowi `RewardFunction` przyszłe implementacje algorytmów DQN oraz Policy Gradient mogą korzystać z wybranej funkcji nagrody bez zależności od jej konkretnej klasy.

Rozwiązanie pozwala również zapisać wariant jako element konfiguracji eksperymentu za pomocą stabilnych wartości tekstowych `net_profit` oraz `stake_scaled_net_profit`.

5.13. Walidacja funkcji nagrody

Testy jednostkowe obejmują wyniki zakończone wygraną gracza, wygraną krupiera, remisem oraz spasowaniem gracza.

Testy integracyjne wykorzystują rzeczywisty przebieg środowiska UTHGame i potwierdzają, że kroki nieterminalne generują nagrodę równą zero, natomiast krok terminalny wykorzystuje rozliczenie zawarte w obiekcie `StepResult`.

Dodatkowo zweryfikowano inwariant

$$R_{\text{scaled}} = \frac{R_{\text{net}}}{6} \quad (5.26)$$

dla różnych wyników rozdania.

Testy potwierdzają również zachowanie znaku, wartości zerowej oraz uporządkowania wyników po zastosowaniu skalowania.

5.14. Hipoteza dotycząca funkcji nagrody

W eksperymentach oba warianty funkcji nagrody zostaną wykorzystane z tymi samymi algorytmami oraz reprezentacjami stanu.

Przyjęto hipotezę, że zmniejszenie skali terminalnego sygnału nagrody może wpływać na stabilność oraz przebieg procesu optymalizacji, szczególnie w metodach wykorzystujących aproksymację funkcji wartości lub gradient polityki.

Nie zakłada się jednak z góry przewagi któregośkolwiek wariantu. Wpływ skali nagrody na jakość oraz szybkość uczenia zostanie określony na podstawie wyników eksperymentalnych.

5.15. Deep Q-Network

Pierwszym algorytmem uczenia przez wzmacnianie zastosowanym w pracy jest Deep Q-Network, dalej określany skrótem DQN. Metoda wykorzystuje sieć neuronową jako aproksymator funkcji wartości akcji $Q(s, a)$. Dla otrzymanej obserwacji model wyznacza wartość oczekiwanego zdyskontowanego zwrotu dla każdej dostępnej akcji [6], [7].

W przeciwieństwie do reprezentacji tablicowej nie jest konieczne przechowywanie osobnej wartości Q dla każdego możliwego stanu. Sieć neuronowa umożliwia generalizację pomiędzy obserwacjami posiadającymi podobne cechy. Jest to szczególnie istotne w badanym środowisku, gdzie liczba możliwych kombinacji kart oraz etapów rozdania uniemożliwia praktyczne zastosowanie pełnej reprezentacji tablicowej.

Implementacja DQN korzysta wyłącznie z obiektu `UTHObservation`. Pełny stan środowiska, w szczególności karty krupiera, karty spalone oraz przyszła kolejność talii,

nie są przekazywane do modelu.

5.15.1. Architektura sieci Q

Sieć wartości została zaimplementowana w klasie `QNetwork`. Liczba neuronów wejściowych zależy od wybranego wariantu reprezentacji stanu. Dla State A wynosi ona 373, natomiast dla State B wynosi 393.

Domyślna architektura zawiera dwie warstwy ukryte po 256 neuronów. Po każdej warstwie ukrytej zastosowano funkcję aktywacji ReLU. Warstwa wyjściowa posiada sześć neuronów odpowiadających wszystkim wartościom typu `Action`.

Architekturę można zatem przedstawić jako

$$d_{\text{state}} \rightarrow 256 \rightarrow 256 \rightarrow 6, \quad (5.27)$$

gdzie

$$d_{\text{state}} \in \{373, 393\}. \quad (5.28)$$

Dla stanu s_t sieć zwraca wektor

$$Q_{\theta}(s_t) = [Q_{\theta}(s_t, a_1), \dots, Q_{\theta}(s_t, a_6)], \quad (5.29)$$

gdzie θ oznacza parametry sieci.

5.15.2. Stałe odwzorowanie akcji

W implementacji przyjęto jedno stałe odwzorowanie wartości `Action` na indeksy wyjścia sieci neuronowej.

Kolejność obejmuje akcje

$$[\text{CHECK}, \text{BET_4X}, \text{BET_3X}, \text{BET_2X}, \text{BET_1X}, \text{FOLD}]. \quad (5.30)$$

Stałość odwzorowania jest wymagana zarówno podczas wyboru akcji, jak i podczas zapisu przejść do bufora doświadczeń oraz wyznaczania wartości używanej w funkcji straty.

5.15.3. Maskowanie nielegalnych akcji

Nie wszystkie sześć akcji jest dostępnych w każdym punkcie decyzyjnym Ultimate Texas Hold'em. Z tego względu samo wybranie największej wartości spośród wszystkich wyjść sieci mogłoby prowadzić do decyzji niedozwolonej przez zasady środowiska.

Dla każdej obserwacji tworzona jest maska

$$m_t \in \{0, 1\}^6, \quad (5.31)$$

gdzie wartość 1 oznacza akcję legalną.

Przed wyznaczeniem maksimum wartości odpowiadające akcjom niedozwolonym są zastępowane wartością $-\infty$. W efekcie

$$a_t = \arg \max_{a \in A(s_t)} Q_\theta(s_t, a), \quad (5.32)$$

gdzie $A(s_t)$ oznacza zbiór legalnych akcji.

Maskowanie wykorzystywane jest w dwóch niezależnych miejscach. Pierwszym jest wybór działania przez agenta. Drugim jest obliczanie celu Bellmana dla następnego stanu.

Drugie zastosowanie jest szczególnie istotne, ponieważ bez niego algorytm mógłby propagować podczas uczenia wysoką wartość Q przypisaną akcji, której wykonanie w danym stanie nie jest możliwe.

5.15.4. Strategia epsilon-greedy

Podczas treningu zastosowano strategię eksploracji epsilon-greedy. Z prawdopodobieństwem ϵ agent wybiera losową akcję spośród akcji legalnych. W pozostałych przypadkach wybierana jest legalna akcja o największej wartości przewidywanej przez sieć Q .

Prawdopodobieństwo eksploracji jest zmniejszane liniowo zgodnie z zależnością

$$\epsilon_t = \epsilon_{\text{start}} + (\epsilon_{\text{end}} - \epsilon_{\text{start}}) \min \left(\frac{t}{T_{\text{decay}}}, 1 \right). \quad (5.33)$$

W konfiguracji bazowej przyjęto

$$\epsilon_{\text{start}} = 1, \quad \epsilon_{\text{end}} = 0,05. \quad (5.34)$$

Po osiągnięciu końca harmonogramu wartość epsilon pozostaje równa ϵ_{end} .

Podczas ewaluacji eksploracja zostaje całkowicie wyłączona przez ustawienie

$$\epsilon = 0. \quad (5.35)$$

Oceniana jest więc deterministyczna polityka zachłanna wynikająca z wyuczonych wartości Q .

5.15.5. Bufor doświadczeń

DQN wykorzystuje mechanizm experience replay [7]. Każda interakcja ze środowiskiem jest zapisywana jako przejście

$$e_t = (s_t, a_t, r_t, s_{t+1}, d_t, m_{t+1}), \quad (5.36)$$

gdzie d_t określa, czy krok zakończył epizod, natomiast m_{t+1} jest maską legalnych akcji następnej obserwacji.

Dla przejścia terminalnego następny stan oraz następna maska nie występują. Zapobiega to przypadkowemu wykonywaniu bootstrapu po zakończeniu rozdania.

Bufor posiada ograniczoną pojemność. Po jej osiągnięciu najstarsze przejścia są zastępowane nowymi. Minibatch jest losowany bez zwracania za pomocą niezależnego generatora liczb pseudolosowych.

5.15.6. Sieć docelowa i cel Bellmana

W implementacji utrzymywane są dwie sieci o identycznej architekturze. Pierwsza z nich, określana jako policy network, jest aktualizowana przez optymalizator. Druga, target network, służy do wyznaczania wartości docelowych.

Dla przejścia terminalnego cel ma postać

$$y_t = r_t. \quad (5.37)$$

Nie następuje wtedy bootstrap z następnego stanu.

Dla przejścia nieterminalnego stosowany jest cel

$$y_t = r_t + \gamma \max_{a' \in A(s_{t+1})} Q_{\theta^-}(s_{t+1}, a'), \quad (5.38)$$

gdzie θ^- oznacza parametry sieci docelowej.

Maksimum jest wyznaczane wyłącznie po akcjach legalnych w następnym stanie. Obliczenie wartości docelowej wykonywane jest bez śledzenia gradientów.

Parametry target network nie są bezpośrednio modyfikowane przez optymalizator. Są okresowo zastępowane parametrami policy network.

5.15.7. Funkcja straty i optymalizacja

Dla każdego przejścia policy network wyznacza wartości wszystkich akcji, lecz funkcja straty wykorzystuje jedynie wartość odpowiadającą akcji faktycznie wykonanej w danym stanie:

$$Q_{\theta}(s_t, a_t). \quad (5.39)$$

Jako funkcję straty zastosowano Smooth L1 Loss, określaną również jako Huber loss:

$$L_t = \mathcal{L}_{\text{Huber}}(Q_{\theta}(s_t, a_t), y_t). \quad (5.40)$$

Parametry policy network są aktualizowane za pomocą optymalizatora Adam. Target network nie należy do parametrów przekazywanych optymalizatorowi.

5.15.8. Przebieg treningu

Pojedynczy epizod treningowy odpowiada jednemu rozdaniu Ultimate Texas Hold'em. Na początku epizodu środowisko generuje nową obserwację preflop.

W każdym kroku wykonywana jest następująca sekwencja:

1. zakodowanie bieżącej obserwacji przez wybrany StateEncoder,
2. ustawienie wartości epsilon wynikającej z aktualnego kroku treningu,
3. wybór legalnej akcji przez agenta,
4. wykonanie akcji w środowisku,
5. wyznaczenie nagrody za pomocą wybranej implementacji RewardFunction,
6. utworzenie przejścia i zapisanie go w buforze doświadczeń,
7. po zakończeniu fazy rozgrzewkowej wylosowanie minibatcha,
8. wykonanie kroku optymalizacji,
9. okresowa synchronizacja sieci docelowej.

Proces trwa do osiągnięcia stanu terminalnego, a następnie rozpoczynane jest kolejne niezależne rozdanie.

5.15.9. Konfiguracja treningu

Parametry treningu są przechowywane w niemutowalnym obiekcie DQNConfig. Konfiguracja wykorzystana w pierwszym pilotażowym uruchomieniu została przedstawiona w tabeli 5.3.

Tabela 5.3. Konfiguracja pilotażowego treningu DQN

Parametr	Wartość
Learning rate	0,001
Gamma	0,99
Batch size	64
Replay capacity	10000
Warmup steps	1000
Target sync interval	1000
Epsilon start	1,0
Epsilon end	0,05
Epsilon decay steps	10000
Warstwy ukryte	256, 256
Epizody treningowe	10000

Konfiguracja ta służyła do weryfikacji kompletnego procesu treningu i nie jest traktowana jako wynik końcowego strojenia hiperparametrów.

5.15.10. Reprodukowalność treningu

W celu ograniczenia niekontrolowanego sprzężenia różnych źródeł losowości z jednego głównego seeda deterministycznie wyprowadzane są osobne seedy dla inicjalizacji PyTorch, generowania rozdań, strategii epsilon-greedy oraz próbkowania bufora doświadczeń.

Oznacza to, że pobranie próbki z replay buffera nie zmienia kolejności przyszłych rozdań, a losowa eksploracja agenta nie zużywa liczb pseudolosowych generatora środowiska.

Testy integracyjne potwierdzają, że dla tej samej konfiguracji i tego samego głównego seeda dwa krótkie uruchomienia treningowe generują identyczne statystyki epizodów oraz identyczne parametry policy network w tym samym środowisku wykonawczym.

Nie zakłada się natomiast bitowej identyczności pomiędzy wszystkimi platformami sprzętowymi oraz wersjami biblioteki PyTorch.

5.15.11. Zapisywanie modeli

Wyuczona policy network może zostać zapisana jako checkpoint. Oprócz parametrów sieci przechowywane są informacje umożliwiające jednoznaczną interpretację modelu:

- wersja formatu checkpointu,
- wymiar wejścia,
- rozmiary warstw ukrytych,
- wariant reprezentacji stanu,
- wariant funkcji nagrody,
- seed treningowy,
- pełna konfiguracja `DQNConfig`.

Podczas odczytu sprawdzana jest zgodność architektury sieci, reprezentacji stanu oraz zapisanej konfiguracji. Checkpoint nie zawiera całego obiektu środowiska ani bufora doświadczeń. Służy przede wszystkim do odtworzenia wyuczonej polityki i przeprowadzenia późniejszej ewaluacji.

5.15.12. Macierz wariantów DQN

Ponieważ implementacja DQN jest niezależna od konkretnego `StateEncoder` oraz `RewardFunction`, możliwe jest utworzenie czterech wariantów:

Pozwala to analizować wpływ reprezentacji oraz skali sygnału nagrody przy zachowaniu tej samej implementacji algorytmu DQN.

Tabela 5.4. Warianty DQN wynikające z reprezentacji stanu i funkcji nagrody

Wariant	Reprezentacja	Nagroda
DQN-A-A	State A	Reward A
DQN-A-B	State A	Reward B
DQN-B-A	State B	Reward A
DQN-B-B	State B	Reward B

Spis rysunków

3.1	Architektura i przepływ informacji w środowisku UTH	22
3.2	Maszyna stanów pojedynczego rozdania UTH	31

Spis tabel

2.1	Ranking układów pokerowych od najsilniejszego do naj słabszego	12
2.2	Porównanie Texas Hold'em i Ultimate Texas Hold'em	13
2.3	Dostępne decyzje gracza w Ultimate Texas Hold'em	13
2.4	Tabela wypłat zakładu Blind przyjęta w modelu środowiska	14
2.5	Przykładowe wartości przewagi kasyna dla wybranych gier kasynowych . .	15
3.1	Podział odpowiedzialności pomiędzy modułami środowiska UTH	21
3.2	Liczba kart przechowywanych w stanie w poszczególnych fazach	27
3.3	Porównanie pełnego stanu środowiska i obserwacji agenta	29
3.4	Legalne akcje w poszczególnych fazach środowiska UTH	31
3.5	Rozliczenie zakładów po showdownie	33
3.6	Tabela wypłat zakładu Blind	35
3.7	Publiczny interfejs klasy UTHGame	38
4.1	Wyniki agentów bazowych dla 10 000 rozdań	47
4.2	Rozkład wyników rozdań agentów bazowych	48
5.1	Porównanie reprezentacji stanu	56
5.2	Porównanie wariantów funkcji nagrody	60
5.3	Konfiguracja pilotażowego treningu DQN	65
5.4	Warianty DQN wynikające z reprezentacji stanu i funkcji nagrody	67

Bibliografia

- [1] Tulalip Resort Casino, *Ultimate Texas Hold'em Game Rules*, https://www.everythingtulalip.com/wp-content/uploads/2024/04/ONEclub_0920_GameRules_RackCard_UltimateTexasHoldem-WEB.pdf, Dostęp: 2026-07-12.
- [2] Nevada Gaming Control Board, *Ultimate Texas Hold'em - Rules of Play*, https://www.gaming.nv.gov/siteassets/content/divisions/enforcement/rules-of-play/Ultimate_Texas_Hold_em.pdf, Dostęp: 2026-07-07.
- [3] W. of Odds, *Ultimate Texas Hold'em*, <https://wizardofodds.com/games/ultimate-texas-hold-em/>, Dostęp: 2026-07-07.
- [4] J. B. Maverick, *Understanding the House Edge: How Casinos Ensure Profit*, <https://www.investopedia.com/articles/personal-finance/110415/why-does-house-always-win-look-casino-profitability.asp>, Dostęp: 2026-07-12.
- [5] Wizard of Odds, *What is the House Edge?* <https://wizardofodds.com/gambling/house-edge/>, Dostęp: 2026-07-12.
- [6] R. S. Sutton i A. G. Barto, *Reinforcement Learning: An Introduction*, 2 wyd. Cambridge, Massachusetts: The MIT Press, 2018.
- [7] V. Mnih i in., „Human-level control through deep reinforcement learning,” *Nature*, t. 518, s. 529–533, 2015. DOI: 10.1038/nature14236